THESIS

El Mokni Mohamed Amin

Debrecen, 2026

University of Debrecen — Faculty of Informatics

Leveraging Agentic AI for Contextual User Flow Discovery and Heuristic Evaluation

Supervisor

Lakatos Róbert

Author

El Mokni Mohamed Amin

MSc in Computer Science

Debrecen, 2026

Contents

Introduction **7**

1 Background and Related Work **9**

1.1 AI Agents and Autonomous Systems 9

1.1.1 From Language Models to Autonomous Agents 9

1.1.2 The ReAct Paradigm 9

1.1.3 LangChain, `create_agent`, and LLM Provider Selection 10

1.2 Web Crawling and Information Extraction 10

1.2.1 Web Crawling Fundamentals 10

1.2.2 Firecrawl 11

1.2.3 The Model Context Protocol 11

1.3 User Flow Discovery 11

1.3.1 Definition and Role in UX Design 11

1.3.2 Traditional Discovery Methods and Their Limitations 12

1.3.3 Motivation for AI-Driven Flow Discovery 12

1.4 Heuristic Evaluation 12

1.4.1 Nielsen's Ten Usability Heuristics 12

1.4.2 The Traditional Expert-Review Process and Its Limitations 12

1.4.3 LLM-Based Approximation of Heuristic Checks 13

1.5 Diagram Generation with Mermaid 14

1.5.1 Text-Based Diagramming and Mermaid 14

1.5.2 Integration in the Agentic Crawler 14

2 System Design and Architecture **15**

2.1 Requirements and Goals 15

2.1.1 Functional Requirements 15

2.1.2 Non-Functional Requirements 16

2.2 High-Level Architecture 16

2.2.1 Component Overview 16

2.2.2 The MCP Connection Lifecycle . . . 17
2.2.3 Concurrency Model . . . 18
2.3 Agent Layer . . . 18
2.3.1 EvaluationAgent . . . 18
2.3.2 LLM Model Factory . . . 19
2.4 Prompt Engineering . . . 20
2.4.1 Definition and Role of Prompt Engineering . . . 20
2.4.2 Structured Analysis Strategy . . . 20
2.4.3 Output Format Specification . . . 20
2.4.4 Heuristic Evaluation Enforcement . . . 20
2.4.5 Mermaid Syntax Hardening . . . 21
2.4.6 Dynamic Analysis Date Binding . . . 21
2.4.7 Do/Don't Rules . . . 21
2.4.8 Temperature and Determinism . . . 21
2.5 Service Layer . . . 21
2.5.1 ChatSessionService . . . 21
2.5.2 CacheService . . . 24
2.5.3 ExecutionTracker . . . 24
2.5.4 Content Normalisation . . . 24
2.6 Data Models . . . 25
2.7 Design Patterns Applied . . . 26

3 Implementation **27**
3.1 Application Interfaces . . . 27
3.1.1 FastAPI HTTP Server . . . 27
3.1.2 Command-Line Interface . . . 28
3.2 Caching and Performance . . . 28
3.2.1 Cache Key Generation . . . 28
3.2.2 Cache Flow . . . 29
3.2.3 Performance Impact . . . 29
3.3 Frontend: React SPA . . . 30
3.4 Diagram Rendering Pipeline . . . 32

4 Report Generation and Evaluation **34**
4.1 Website Analysis Report Structure . . . 34
4.2 Contextual User Flow Discovery . . . 36
4.2.1 How the Agent Identifies User Journeys . . . 36
4.2.2 Examples . . . 36

4.3 Heuristic Evaluation Output . . . 38
4.3.1 Dedicated Heuristic Evaluation Section . . . 38
4.3.2 Observable Signals and Grounding . . . 39
4.3.3 Limitations of the Automated Approach . . . 40
4.4 System Evaluation . . . 41
4.4.1 Evaluation Methodology . . . 41
4.4.2 Site A: Faculty of Informatics, University of Debrecen . . . 42
4.4.3 Site B: web-scraping.dev/products . . . 42
4.4.4 Comparative Observations . . . 43
4.4.5 Token Usage and Estimated Cost . . . 43
4.4.6 Cache Effectiveness . . . 44
4.4.7 Known Trade-offs . . . 45

5 Challenges and Limitations 46
5.1 Technical Challenges . . . 46
5.1.1 Non-Streaming HTTP Requests . . . 46
5.1.2 Browser Session Scope . . . 46
5.1.3 No Support for SPAs and Authentication-Gated Content . . . 47
5.2 Scalability Constraints . . . 47
5.3 Security Considerations . . . 47

6 Future Work 49
6.1 Planned Enhancements . . . 49
6.1.1 Streaming Responses . . . 49
6.1.2 Distributed Cache and Session Store . . . 50
6.1.3 Diagram Export . . . 50
6.1.4 Scheduled Analyses and Change Detection . . . 50
6.1.5 Version Comparison Reports . . . 50
6.1.6 Authentication Layer . . . 51
6.1.7 Additional LLM Provider Support . . . 51
6.1.8 Interactive Clickable Diagrams . . . 51

Conclusion 52

A API Endpoint Reference 58

B Sample Generated Website Analysis Report 61

C Environment Variables and Deployment 66

D Project File Structure **69**

E System Prompt **72**

Introduction

The World Wide Web has grown into a vast and complex ecosystem, with billions of websites spanning every domain of human activity. For developers, UX researchers, and product teams, understanding the structure, content, and navigational logic of a website is a routine yet time-consuming task. A thorough manual analysis—mapping pages, tracing user journeys, and evaluating usability—can take hours even for a medium-sized site, and the results are inherently subjective and difficult to reproduce.

Recent advances in artificial intelligence have opened a new path forward. Large Language Models (LLMs) such as Google Gemini and Meta's LLaMA can now reason, plan, and orchestrate external tools autonomously, giving rise to so-called *agentic AI systems*. Unlike traditional software that follows a fixed sequence of steps, an agent adapts its strategy dynamically based on what it discovers—a property that makes it well-suited to open-ended analytical tasks like website exploration.

Despite this potential, no existing tool combines web crawling and LLM-based reasoning into a fully automated pipeline for user flow discovery and heuristic evaluation. Analytics platforms provide traffic data but no structural insight; usability testing tools require human experts; static scanners produce technical reports without semantic understanding. This thesis addresses that gap, and investigates the following research question: *can an agentic AI system autonomously discover contextual user flows and perform a heuristic evaluation of web interfaces, producing structured, actionable output without manual intervention?*

To answer this question, the **Agentic Crawler** was designed and implemented. The system uses LangChain's agent framework together with Firecrawl—a web crawling service integrated via the Model Context Protocol (MCP)—to autonomously map a target website, scrape key pages, and synthesise the findings into a structured analysis report. Reports include an executive summary, sitemap diagrams, and user flow diagrams in Mermaid syntax, alongside design and UX observations grounded in Nielsen's ten usability heuristics [13]. The system is accessible through both an interactive CLI and a FastAPI REST API, the latter consumed by a React single-page application that renders the agent's output—including live Mermaid diagrams—in the browser. Both interfaces share a common `ChatSessionService` layer, ensuring consistent behaviour regardless of entry point. The remainder of this thesis is structured as follows. Chap-

ter 1 covers the relevant background: agentic AI and the ReAct paradigm, web crawling and MCP, user flow concepts, Nielsen's heuristic evaluation framework, and Mermaid diagram generation. Chapter 2 presents the system design, and Chapter 3 details its implementation across the backend, frontend, and caching subsystem. Chapter 4 evaluates the quality and performance of the system's output, while Chapters 5 and 6 address limitations and planned enhancements respectively. The thesis closes with a conclusion reflecting on the findings and their broader implications for the automation of UX analysis workflows. The full source code is publicly available at `https://github.com/SOURON/thesis-work`.

Chapter 1

Background and Related Work

This chapter establishes the theoretical and technical foundations on which the Agentic Crawler is built, covering agentic AI and the ReAct paradigm (Section 1.1), web crawling techniques and the Model Context Protocol (Section 1.2), user flow discovery (Section 1.3), Nielsen's heuristic evaluation framework (Section 1.4), and diagram generation with Mermaid (Section 1.5).

1.1 AI Agents and Autonomous Systems

1.1.1 From Language Models to Autonomous Agents

The rise of Large Language Models (LLMs) such as GPT-4 [16], Google Gemini [7], and Meta's LLaMA [24] has fundamentally changed what software systems can do. While early language models were designed to perform a single task—predicting the next token in a sequence—modern LLMs exhibit emergent capabilities such as multi-step reasoning, code generation, and instruction following that extend far beyond simple text completion.

Building on these capabilities, researchers and engineers began constructing *AI agents*: systems that do not merely respond to a single prompt, but perceive their environment, maintain a goal, decide which actions to take, and iterate until that goal is achieved. An agent is a closed loop: it receives observations, reasons about what to do next, executes an action by calling an external tool, observes the result, and reasons again until the task is complete [29].

1.1.2 The ReAct Paradigm

A particularly influential formalisation of this loop is the *ReAct* framework introduced by Yao et al. [31]. ReAct—short for **Re**asoning and **Act**ing—interleaves two alternating step types. A **Thought** is a free-form reasoning step in which the model reflects on the current state and plans which action to take next. An **Action** is a structured call to an external tool together with

its arguments. After each action the environment returns an **Observation**—the tool's output—which the model incorporates into its next thought. This cycle grounds the agent's reasoning in retrieved information rather than in knowledge encoded solely in its weights, which is essential for tasks requiring up-to-date or domain-specific data.

1.1.3 LangChain, `create_agent`, and LLM Provider Selection

Implementing the ReAct loop from scratch requires careful management of message history, tool schemas, response parsing, and error recovery. The LangChain framework [9] abstracts this complexity through a standardised agent execution engine. Its `create_agent` function accepts an LLM instance, a list of tools, and an optional system prompt, and returns a runnable agent that manages the Thought–Action–Observation loop internally, exposing a clean `invoke` / `ainvoke` interface to the caller.

In the Agentic Crawler this abstraction is used inside the `EvaluationAgent` class, which supports two LLM providers selected via configuration: **Google Gemini** (`gemini-2.5-flash-lite`, default) accessed through the `ChatGoogleGenerativeAI` integration, and **Groq / LLaMA** (`llama-3.3-70b-versatile`) via Groq's inference API. Because LangChain returns a standard `BaseChatModel` instance in both cases, the rest of the system is completely provider-agnostic. The tool list is populated at runtime from the Firecrawl MCP server, so the agent has no hard-coded knowledge of which tools are available—it discovers them dynamically at connection time. Provider instantiation and the Factory pattern it employs are discussed in Section 2.3.

1.2 Web Crawling and Information Extraction

1.2.1 Web Crawling Fundamentals

Web crawling is the automated process of systematically browsing the World Wide Web to collect and index content. A crawler begins at one or more seed URLs, retrieves the corresponding pages, extracts hyperlinks, and enqueues them for subsequent retrieval, repeating until a stopping criterion is met [15]. Two classical traversal strategies govern page ordering: *breadth-first crawling* processes all links at a given depth before advancing to the next level, making it well-suited to capturing top-level site structure quickly, while *depth-first crawling* follows a single link chain as far as possible before backtracking [3]. Modern crawling tasks also require *content extraction*—isolating meaningful text from surrounding markup, advertisements, and boilerplate—using techniques ranging from DOM parsing to machine-learning-based content detection [4].

1.2.2 Firecrawl

The Agentic Crawler delegates all web interaction to **Firecrawl** [10], a managed crawling service that exposes three operations: `firecrawl_map` discovers all URLs within a domain and is always invoked first; `firecrawl_scrape` retrieves and cleans a single page as Markdown, with an `onlyMainContent` flag to strip boilerplate; and `firecrawl_crawl` performs a bulk multi-page crawl up to a configurable limit. Firecrawl handles JavaScript rendering, rate limiting, and anti-bot measures transparently, allowing the agent to treat web content as a uniform stream of clean Markdown.

1.2.3 The Model Context Protocol

Each LLM API has historically defined its own tool-calling format, requiring bespoke adapter code for every model-service combination. The **Model Context Protocol** (MCP), introduced by Anthropic in 2024 [1], addresses this by defining an open JSON-RPC-based protocol through which any host application can discover, describe, and invoke tools exposed by any compliant server. An MCP server publishes a manifest of tools, each described by a name, natural-language description, and JSON Schema input definition. The client queries this manifest at connection time and presents the tool schemas to the LLM as part of its context.

In the Agentic Crawler, Firecrawl runs as an MCP server launched as a local subprocess via `npx firecrawl-mcp`. The connection is established once at application startup and held open for the entire process lifetime, avoiding repeated initialisation overhead. The tools loaded from this connection are passed directly to `EvaluationAgent`, so the agent discovers the available Firecrawl operations dynamically through the protocol rather than through hard-coded knowledge. The full connection lifecycle implementation is described in Section 2.2.

1.3 User Flow Discovery

1.3.1 Definition and Role in UX Design

A *user flow* is a representation of the sequence of steps a user takes to accomplish a specific goal within a digital interface [6]. It captures the path from an entry point—such as a landing page or a search result—through a series of intermediate screens or states, to a terminal outcome such as a completed purchase, a submitted form, or the consumption of a piece of content. User flows are distinct from sitemaps in that they are goal-oriented and dynamic: the same set of pages may participate in multiple flows depending on the user's intent. In UX practice they serve as design artefacts, evaluation baselines, and documentation tools that communicate intended behaviour to developers and stakeholders [25].

1.3.2 Traditional Discovery Methods and Their Limitations

Discovering user flows from an existing website has traditionally required substantial manual effort. **Expert walkthroughs** produce high-quality annotated flows but do not scale to sites with many distinct journeys. **Session recording and analytics** tools such as Hotjar or FullStory ground findings in real usage data [2], but require live-site instrumentation and sufficient traffic. **Collaborative workshops** such as user story mapping [17] align teams but depend on domain expert availability and are unsuitable for analysing third-party sites.

All three approaches share a common limitation: they require either direct access to a live product's analytics infrastructure or the sustained attention of a skilled human analyst—neither of which is available when the goal is to quickly understand an unfamiliar website.

1.3.3 Motivation for AI-Driven Flow Discovery

An agent equipped with crawling tools can autonomously explore a website's navigational structure, infer the purpose of each page from its content, and reconstruct likely user journeys without instrumentation, live traffic, or human annotation [8]. The Agentic Crawler operationalises this idea: its system prompt instructs the agent to identify entry points, conversion points, and decision points from the crawled content, and to represent each key journey as a Mermaid `flowchart TD` diagram with explicit branching. This approach trades the precision of analytics-derived flows for breadth and accessibility—any publicly accessible website can be analysed in minutes, producing visual flow documentation that would otherwise require hours of manual work. The trade-offs of this approximation are discussed in Section 4.3.

1.4 Heuristic Evaluation

1.4.1 Nielsen's Ten Usability Heuristics

Heuristic evaluation is a usability inspection method in which one or more evaluators examine a user interface and judge its compliance against a set of recognised usability principles, known as heuristics [13]. The method was introduced by Nielsen and Molich in 1990 and subsequently refined into the ten heuristics that have become the de facto standard for usability auditing in both industry and academia [14]. Table 1.1 summarises the ten heuristics.

1.4.2 The Traditional Expert-Review Process and Its Limitations

In classical practice, a heuristic evaluation is conducted by a small panel of usability experts—Nielsen recommends three to five evaluators—each of whom independently inspects the interface and records violations against the ten heuristics, assigning a severity rating to each finding

H#	Heuristic
H1	Visibility of system status
H2	Match between system and the real world
H3	User control and freedom
H4	Consistency and standards
H5	Error prevention
H6	Recognition rather than recall
H7	Flexibility and efficiency of use
H8	Aesthetic and minimalist design
H9	Help users recognise, diagnose, and recover from errors
H10	Help and documentation

Table 1.1: Nielsen's ten usability heuristics [13].

[13]. Individual reports are then aggregated to produce a consolidated list of issues ranked by severity.

While this process is one of the most cost-effective usability methods available [26], it carries several inherent limitations. It requires access to trained usability professionals, making it expensive and difficult to scale. It is inherently subjective—different evaluators may reach different conclusions about the same interface element [23]. It is time-intensive, and it is not easily repeatable: evaluating a later version of a site to track improvements requires a fresh manual review session.

1.4.3 LLM-Based Approximation of Heuristic Checks

The convergence of LLM reasoning capabilities and automated web crawling opens the possibility of approximating heuristic evaluation programmatically. Recent work has demonstrated that large language models possess substantial latent knowledge of usability principles and can generate meaningful heuristic assessments when appropriately prompted [23, 8].

The Agentic Crawler operationalises this through its *Design & UX Observations* and *Technical Observations* report sections, generated from the Markdown content returned by Firecrawl. From this content the agent can reason about H4 (consistency of navigation labels and heading structure), H6 (whether primary actions are visible without requiring prior recall), H8 (content density and typographic hierarchy), and H10 (presence of FAQ pages, help centres, and onboarding guides). Heuristics involving dynamic interface states—such as H1 (system status feedback) and H5 (error prevention)—are harder to assess from static content, though the agent can note the presence or absence of loading indicators and form validation cues as proxy signals.

This approach constitutes an *approximation* rather than a full heuristic evaluation: it is grounded in static content analysis rather than live interaction testing, and it cannot detect dynamic interface states or time-based feedback. The scope and fidelity of this approximation are

discussed further in Section 4.3.

1.5 Diagram Generation with Mermaid

1.5.1 Text-Based Diagramming and Mermaid

Visual diagrams are among the most effective tools for communicating the structure and behaviour of software systems and user interfaces. Traditional graphical editors such as Lucidchart or draw.io produce polished results but require manual interaction, which is incompatible with an automated LLM-driven pipeline: a language model can only produce text, not operate a GUI tool.

Text-based diagramming systems solve this by representing diagrams as structured plain-text definitions rendered into vector graphics by a separate engine. Mermaid [11] is the leading open-source framework of this kind. It supports a concise, readable syntax, is natively rendered by GitHub, GitLab, Notion, and Obsidian, and produces SVG output that is crisp at any resolution.

The Agentic Crawler uses two Mermaid diagram types in its generated reports. **Graph diagrams** (`graph TD`) represent hierarchical page structures as nodes and directed edges, and are used for sitemaps. **Flowchart diagrams** (`flowchart TD`) extend this with decision nodes, rounded terminal nodes, and conditional edge labels, making them the appropriate choice for user flow diagrams where branching logic and conversion points must be clearly expressed.

1.5.2 Integration in the Agentic Crawler

On the backend, the agent's system prompt instructs it to embed all diagrams as fenced Mermaid code blocks inside the Markdown report, so diagram specifications travel through the system as plain text with no special serialisation.

On the frontend, the `MarkdownMessage` component detects fenced blocks annotated with the `mermaid` language tag and routes them to the dedicated `MermaidBlock` component, which calls `mermaid.render()` asynchronously and injects the resulting SVG into the DOM. If rendering fails, the component falls back to displaying the raw diagram source. The full implementation of this rendering pipeline is described in Section 3.4.

Chapter 2

System Design and Architecture

This chapter describes the design decisions and architectural structure of the Agentic Crawler. It opens with a statement of the system's functional and non-functional requirements (Section 2.1), then presents the high-level component model and the shared orchestration path common to both the CLI and the HTTP API (Section 2.2). The agent layer, including the `EvaluationAgent` class and the LLM model factory, is described in Section 2.3, followed by a dedicated account of the system prompt design and the prompt engineering decisions that shape the agent's output (Section 2.4). The service layer — covering `ChatSessionService`, `CacheService`, `ExecutionTracker`, and the content normalisation module — is examined in Section 2.5. Section 2.6 documents the Pydantic data models that define the system's internal and external data contracts. The chapter closes with an account of the six software design patterns applied throughout the codebase (Section 2.7).

2.1 Requirements and Goals

2.1.1 Functional Requirements

The Agentic Crawler was designed around a core set of functional requirements that define what the system must be capable of doing.

Automated website crawling and analysis. Given a target URL, the system must autonomously discover the site's page structure, retrieve and clean the content of key pages, and synthesise its findings—without any step-by-step human guidance. The agent must decide for itself which pages to visit, in what order, and when it has gathered sufficient information to produce a report.

Structured report generation. The system must produce a consistent, structured Markdown report comprising nine sections: an executive summary, a website summary, a content overview, a sitemap diagram, user flow diagrams, a site structure breakdown, navigation pat-

terns, design and UX observations, and technical observations. Reports that match a recognisable report pattern must be automatically saved to the `reports/` directory.

Mermaid diagram generation. The system must produce valid Mermaid `graph TD` and `flowchart TD` diagrams as part of every report, embedded as fenced code blocks in the Markdown output.

Session memory. Both interfaces must support multi-turn conversations, allowing the user to ask follow-up questions about a previously analysed site within the same session. Message history must persist across turns and be keyed by a stable session identifier.

Dual interface. The same agent orchestration logic must be accessible through two independent entry points: an interactive command-line interface (CLI) for terminal-based use, and a RESTful HTTP API for programmatic and browser-based clients.

2.1.2 Non-Functional Requirements

Beyond functional correctness, several non-functional requirements governed the system's design.

LRU caching with TTL. To reduce redundant and costly Firecrawl and LLM API calls, tool results must be cached using a Least Recently Used (LRU) eviction strategy with a configurable time-to-live (default: 24 hours) and a configurable maximum size (default: 100 MB).

Asynchronous I/O. All agent invocations and tool calls must be non-blocking, using Python's `asyncio` throughout the service layer to avoid holding threads during long-running crawl operations.

Type safety. All data exchanged between components must be validated at runtime using Pydantic models. This includes API request and response bodies, tool call records, execution statistics, and cache statistics.

API key security. Firecrawl, Gemini, and Groq API keys must remain strictly server-side. The browser client must never have access to these credentials; all crawling and LLM inference must occur on the server process.

CORS configurability. The HTTP API must support configurable Cross-Origin Resource Sharing (CORS) policies to allow the React frontend to communicate with the backend during development while permitting restriction to specific origins in production deployments.

2.2 High-Level Architecture

2.2.1 Component Overview

The Agentic Crawler follows a *layered architecture* with two independent entry points that converge on a single shared orchestration path. The two entry points—the CLI and the FastAPI

HTTP server—each initialise the same set of services and delegate all natural-language processing to the same `ChatSessionService`. This design ensures that the behaviour of the system is identical regardless of how it is accessed, and that new entry points can be added in the future without modifying the core logic.

The orchestration path is as follows:

`CLI / FastAPI` → `ChatSessionService` → `EvaluationAgent` → `Firecrawl MCP + LLM`

Each layer has a single, well-defined responsibility. The entry-point layer handles interface-specific concerns—terminal rendering and slash commands for the CLI, HTTP routing and JSON serialisation for the API. The `ChatSessionService` manages per-session message history, tool event collection, cache interaction, execution tracking, and report persistence. The `EvaluationAgent` wraps LangChain's `create_agent` and handles the Thought–Action–Observation loop. The external services layer consists of the Firecrawl MCP server and the chosen LLM provider (Gemini or Groq).

Figure 2.1 illustrates the full component structure and the relationships between layers.

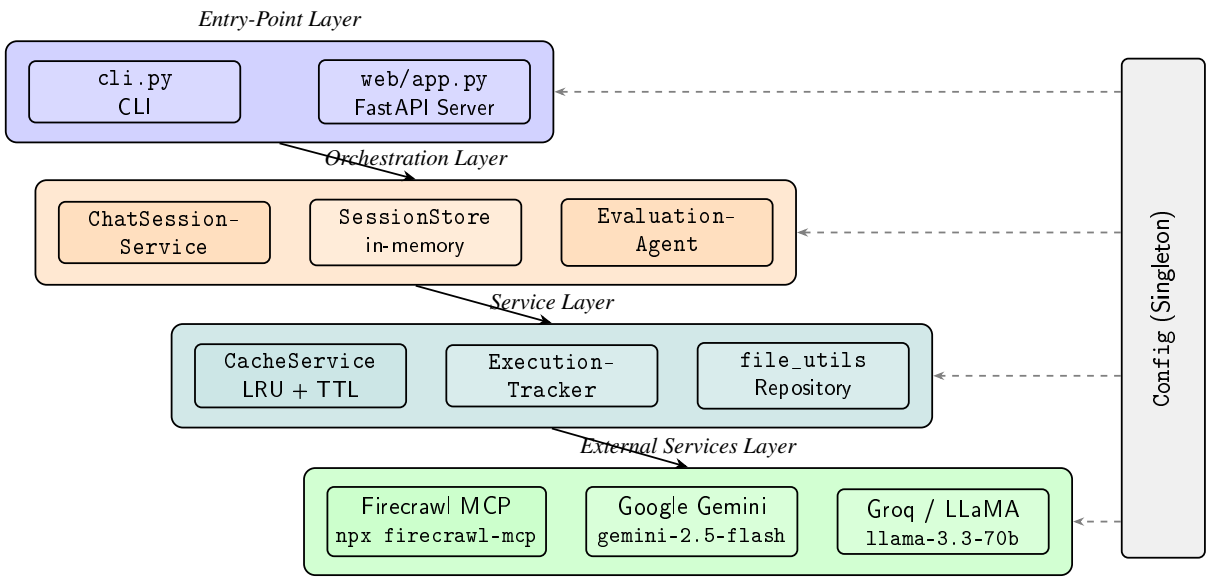


Figure 2.1: High-level component architecture of the Agentic Crawler. Solid arrows indicate the top-to-bottom orchestration flow between layers. Dashed arrows indicate the cross-cutting `Config` singleton, which is accessed by every layer.

2.2.2 The MCP Connection Lifecycle

A central architectural constraint is that the Firecrawl MCP server runs as a subprocess communicating over standard I/O (`stdio`). This means the connection is established once per process lifetime and held open for the duration of the application's execution. Establishing a new MCP

connection for every request would incur a significant handshake overhead, as the subprocess must be spawned, the MCP session initialised, and the tool manifest loaded each time.

In the CLI, this lifecycle is managed through Python's `asynccontextmanager`: the `stdio_client` and `ClientSession` context managers are entered at application startup, the MCP tools are loaded and passed to `EvaluationAgent`, and the connection is cleanly closed when the user exits. In the HTTP API, the same pattern is implemented using FastAPI's `lifespan` hook, which opens the MCP connection before the server begins accepting requests and closes it during shutdown. The agent and all shared services are stored in `app.state` so that request handlers can access them without re-instantiation on every call.

A consequence of this design is that the HTTP API is *not ready* until the MCP connection has been fully initialised. The `GET /api/health` endpoint communicates this to clients by returning `{"ready": false}` until the lifespan hook completes, and the `POST /api/chat` endpoint returns HTTP 503 if it is invoked before readiness is confirmed.

2.2.3 Concurrency Model

The system uses Python's `asyncio` event loop throughout. All agent invocations are performed with `await agent.ainvoke(...)`, and all Firecrawl tool calls are non-blocking. Within `ChatSessionService`, concurrent requests to the same session are serialised using a per-session `asyncio.Lock`, stored in a `defaultdict`. This prevents interleaved writes to the session's message history in the event that two requests arrive simultaneously for the same `session_id`. Different sessions are fully independent and execute concurrently.

2.3 Agent Layer

2.3.1 EvaluationAgent

The `EvaluationAgent` class is the central component of the agent layer. Its constructor accepts three arguments: a list of MCP tools, a model type string (`"google"` or `"groq"`), and an optional custom system prompt. It instantiates the LLM via a factory method `_create_model()` and then calls LangChain's `create_agent` to assemble the full agent:

```
self.agent = create_agent(
    self.model,
    self.tools,
    system_prompt=self.system_prompt
)
```

The resulting agent handles the Thought–Action–Observation loop internally. Callers interact with it exclusively through the `ainvoke(messages)` method, which accepts the full conversation history as a list of role-content dictionaries and returns the complete response including all intermediate tool messages. This stateless interface makes the agent straightforward to test and easy to share between the CLI and HTTP API through `ChatSessionService`. The system prompt that governs the agent's analytical behaviour is discussed in Section 2.4.

2.3.2 LLM Model Factory

The `_create_model()` factory method reads the `model_type` attribute and instantiates the appropriate LangChain chat model:

```
def _create_model(self):
    if self.model_type == "groq":
        return ChatGroq(
            model=self.config.GROQ_MODEL,
            temperature=self.config.MODEL_TEMPERATURE,
            streaming=self.config.ENABLE_STREAMING,
        )
    else:
        return ChatGoogleGenerativeAI(
            model=self.config.GOOGLE_MODEL,
            temperature=self.config.MODEL_TEMPERATURE,
            streaming=self.config.ENABLE_STREAMING,
        )
```

The two supported providers are Google Gemini (`gemini-2.5-flash-lite`, the default) and Groq's LLaMA (`llama-3.3-70b-versatile`). Both are configured with a temperature of 0.3, which balances creativity with consistency in the structured output the system expects. Because the factory returns a standard LangChain `BaseChatModel` instance in both cases, the rest of the agent layer is completely provider-agnostic. Adding support for a third provider requires only a new branch in this method, with no changes to `EvaluationAgent`, `ChatSessionService`, or any other component.

2.4 Prompt Engineering

2.4.1 Definition and Role of Prompt Engineering

Prompt engineering is the practice of designing natural-language instructions that guide an LLM's behaviour without modifying its parameters [21]. Prompts can range from simple task descriptions to elaborate multi-part instructions with role definitions, step-by-step procedures, output templates, and constraint rules [22]. In the Agentic Crawler, the system prompt defines both the analytical strategy (tool call ordering, information gathering, reasoning) and the exact output structure (section headings, diagram syntax, evaluation format). The full prompt is reproduced in Appendix E.

2.4.2 Structured Analysis Strategy

Providing step-by-step task decomposition significantly improves LLM output quality [28]. Without explicit sequencing, early experiments showed the agent skipping site discovery or producing reports before examining enough pages. The prompt enforces a six-step strategy: (1) Discover the site via `firecrawl_map`, (2) Prioritise important pages, (3) Scrape them, (4) Analyse purpose, content, and journeys, (5) Generate Mermaid diagrams, (6) Compile the final report. This ordering ensures the agent always has a complete picture before producing output.

2.4.3 Output Format Specification

Explicit output templates are among the most effective techniques for controlling LLM output shape [21]. The prompt embeds a complete report template with exact section headings (including emoji prefixes for the frontend), placeholder descriptions, and concrete Mermaid examples for both `graph TD` sitemaps and `flowchart TD` user flows. The agent is forbidden from adding, removing, or reordering sections, and must emit raw Markdown rather than wrapping the report in a fenced code block (which would disable rendering in the React frontend).

2.4.4 Heuristic Evaluation Enforcement

The Heuristic Evaluation section requires the agent to assess the target website against all ten of Nielsen's heuristics [13], producing a five-column Markdown table (heuristic, rating, evidence, impact, recommendation) with exactly ten rows. Early experiments showed models frequently truncating after five or six rows or replacing remaining rows with prose. The prompt addresses this with both positive mandates and explicit prohibitions against truncation.

2.4.5 Mermaid Syntax Hardening

Unquoted label text containing slashes, apostrophes, or parentheses caused recurring parse failures in Mermaid diagrams. The prompt includes dedicated guidelines mandating double-quoted strings in all node shapes and edge labels (e.g. `A["Homepage"]`, `-->"Yes"||`), even when labels appear safe. This rule eliminated the majority of rendering failures.

2.4.6 Dynamic Analysis Date Binding

The analysis date in the Executive Summary is injected at construction time by `get_system_prompt()`, which inserts the current UTC timestamp both into the template and as a binding rule instructing the model to reproduce it verbatim. This prevents the model from hallucinating a different date.

2.4.7 Do/Don't Rules

The prompt concludes with explicit rules targeting failure modes observed during development [22]: always use `firecrawl_map` first; use valid Mermaid syntax with quoted labels; include actual discovered URLs; do not dump raw tool output; do not add sections outside the template; emit raw Markdown only; do not truncate the heuristic table.

2.4.8 Temperature and Determinism

The agent uses a temperature of 0.3 for both Gemini and Groq. A temperature of 0.0 produces formulaic prose across different sites; above 0.5 the structured output format is occasionally violated. At 0.3 the agent produces natural, site-specific prose while reliably following the template.

2.5 Service Layer

2.5.1 ChatSessionService

`ChatSessionService` is the orchestration hub shared by both the CLI and the HTTP API. Its `process_turn` method encapsulates the complete lifecycle of a single user turn: appending the user message to the session history, invoking the agent, collecting tool events, updating the cache, tracking token usage, optionally saving the report to disk, and returning a structured `TurnResult` to the caller.

Session history is managed by an embedded `SessionStore`—a thin wrapper around a dictionary that maps `session_id` strings to lists of role-content message dictionaries. The store

is entirely in-memory, which means it does not survive process restarts. Each `process_turn` call reads the current history, appends the user message, calls the agent with the full history as context, and—on success—appends the assistant's reply and writes the updated history back to the store.

Concurrent writes to the same session are prevented by a per-session `asyncio.Lock` stored in a `defaultdict`:

```
self._locks: Dict[str, asyncio.Lock] = defaultdict(lambda:
↪  asyncio.Lock())

async with self._locks[session_id]:
    ...  # read, invoke, write
```

This ensures that if two requests arrive simultaneously for the same session, the second will wait for the first to complete before accessing the message history, preventing race conditions and history corruption. Sessions with different identifiers are unaffected and execute concurrently.

Figure 2.2 shows the complete data flow for a single user turn, from the moment a message enters the system to the moment the structured `TurnResult` is returned.

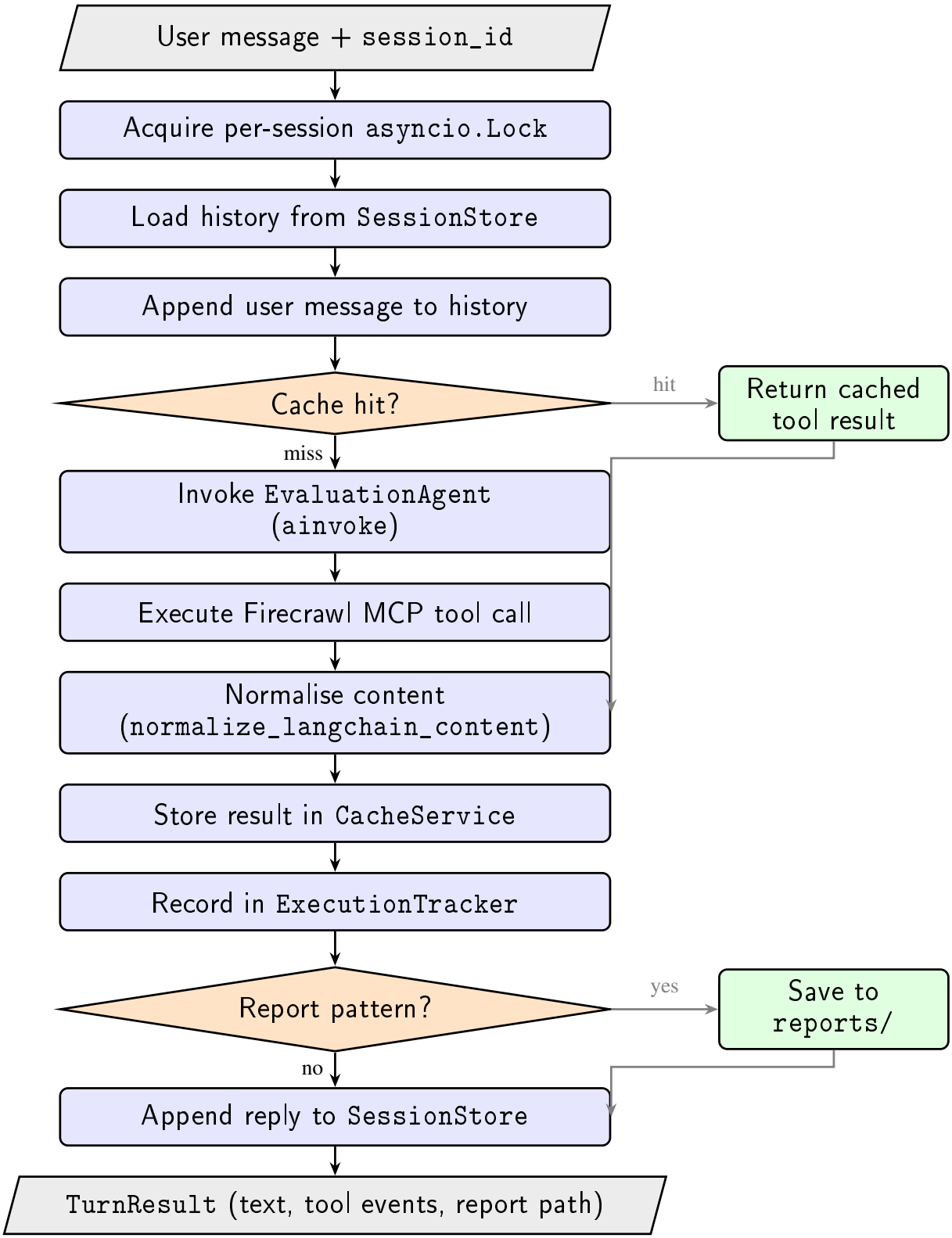


Figure 2.2: Data flow for a single user turn through `ChatSessionService`. Trapezoid nodes represent system input and output; rounded rectangles represent processing steps; diamonds represent decision points. Grey side arrows show the cache-hit and report-save branch paths.

2.5.2 CacheService

`CacheService` implements a combined LRU eviction and TTL expiry strategy for tool call results. Internally it uses Python's `OrderedDict`, which maintains insertion order and supports O(1) move-to-end operations, making it an efficient backing structure for LRU semantics.

Cache keys are generated by computing the MD5 hash of the tool name concatenated with a JSON-serialised, key-sorted representation of the tool arguments:

```
args_str = json.dumps(args, sort_keys=True)
key = hashlib.md5(f"{tool_name}:{args_str}".encode()).hexdigest()
```

Sorting the argument keys before serialisation ensures that two calls with identical arguments in different dictionary orderings produce the same cache key. On a cache hit, the entry is moved to the end of the `OrderedDict` to record it as recently used. If the entry's timestamp is older than the configured TTL, it is treated as expired and removed before a fresh tool call is made. On a cache miss, the tool result is stored with the current timestamp, and entries are evicted from the front of the `OrderedDict` (least recently used) until the total cache size falls below the configured maximum.

2.5.3 ExecutionTracker

`ExecutionTracker` records statistics for every tool call and every agent turn. For each tool invocation, it records the tool name, arguments, start and end time, output length, whether the result was served from cache, and the path of any saved output file. These per-call records are accumulated as `ToolCall` Pydantic model instances.

Token usage is recorded per turn using a heuristic approximation—four characters per token—applied to the serialised message list and the final response string. While not a substitute for provider-reported token counts, this approximation provides a cost estimate that is sufficient for monitoring purposes. The cost is calculated using configurable per-1000-token rates for input and output tokens, and the running total is exposed through the `GET /api/stats` endpoint and the CLI's `/stats` command.

2.5.4 Content Normalisation

A practical challenge when building LangChain pipelines that span multiple LLM providers is that different providers may return message content in different formats. While some providers always return a plain string, others may return a list of typed content blocks—for example, `[{"type": "text", "text": "..."}]`—to support multimodal responses.

The content_normalize.py module addresses this with a single normalisation function, normalize_langchain_content(), that accepts any of these representations and always returns a plain string:

```
if isinstance(content, str):
    return content
if isinstance(content, list):
    parts = []
    for block in content:
        if isinstance(block, dict) and "text" in block:
            parts.append(str(block["text"]))
        ...
    return "\n".join(parts)
return str(content)
```

This normalisation is applied consistently in two places: in CacheService before storing a result (since len(...encode("utf-8")) requires a string), and in ChatSessionService before appending the assistant's reply to the message history. Without this step, a provider switch from Gemini to Groq—or a future provider that returns list-shaped content—would cause silent failures in the cache and incorrect history entries.

2.6 Data Models

All data exchanged between components is represented as Pydantic [18] models, providing runtime type validation, JSON serialisation, and automatic OpenAPI schema generation via FastAPI.

The HTTP API's chat endpoint returns a TurnResult containing the agent's Markdown response, an ordered list of ToolEvent records (tool name, arguments, cached flag, duration, raw output), an optional error message, and the filesystem path of any saved report. Internally, ExecutionTracker records each tool call as a ToolCall model and aggregates session-wide totals into ExecutionStats (token counts, estimated cost, file counts), while CacheService exposes its state through CacheStats (hit/miss counts, entry count, size in megabytes, computed hit rate). The full field-level reference for all API-facing models is available in the OpenAPI schema served by FastAPI and summarised in Appendix A.

2.7 Design Patterns Applied

The Agentic Crawler's architecture draws on several classical design patterns [5]. Table 2.1 summarises each pattern, the component that realises it, and the design problem it solves.

Pattern	Component	Rationale
Singleton	`Config` via `get_config()`	Single source of truth for all environment-derived settings; a `reset_config()` function clears the instance between test runs.
Service Layer	`ChatSessionService`, `CacheService`, `ExecutionTracker`	All orchestration logic lives in the service package; the CLI and HTTP API are thin adapters with no business logic, so changes to orchestration (e.g. adding streaming) require no interface-layer modifications.
Repository	`file_utils.py`	All file system interactions (saving reports, listing outputs) are encapsulated behind a uniform API, making the persistence strategy replaceable without touching consumers.
Factory	`EvaluationAgent._create_llm()`	Isolates provider-specific constructors (`ChatGoogleGenerativeAI` vs. `ChatGroq`) from the agent, which works with the abstract `BaseChatModel` interface. Adding a new provider is a localised change.
Strategy	`CacheService` (LRU eviction)	Eviction policy is decoupled from cache lookup and insertion; substituting LFU or FIFO requires changing only the eviction implementation.
Dependency Injection	`ChatSessionService` constructor	Collaborators (agent, cache, tracker) are injected at construction time, enabling mock injection in tests and guaranteeing identical behaviour across the CLI and HTTP entry points.

Table 2.1: Design patterns applied in the Agentic Crawler architecture.

Chapter 3

Implementation

This chapter describes the concrete implementation of each major component of the Agentic Crawler. It begins with the two application interfaces—the FastAPI HTTP server and the command-line interface—and their shared startup and session handling behaviour (Section 3.1). Section 3.2 details the LRU and TTL caching subsystem, including key generation and the eviction flow. The React single-page application, its state management, API client, and session persistence strategy are covered in Section 3.3. Section 3.4 explains how Mermaid diagrams embedded in the agent's Markdown output are detected and rendered live in the browser. The chapter concludes with a description of the system's observability features and frontend UX design (Section 3.3).

3.1 Application Interfaces

The Agentic Crawler exposes its agent orchestration logic through two independent entry points that share the same `ChatSessionService` core: an HTTP REST API built with FastAPI, and an interactive command-line interface.

3.1.1 FastAPI HTTP Server

The HTTP backend is implemented as a FastAPI [19] application defined in `web/app.py`. FastAPI was chosen for its first-class support for asynchronous request handling, automatic OpenAPI documentation generation from Pydantic models, and its `lifespan` mechanism, which provides structured startup and shutdown hooks that map cleanly onto the MCP connection lifecycle described in Section 2.2.

The lifespan handler is an `asynccontextmanager` that runs before the server accepts its first request. It validates the configuration, spawns the Firecrawl MCP subprocess via `stdio_client`, initialises the `ClientSession`, loads the MCP tools, and constructs the full service graph—

EvaluationAgent, CacheService, ExecutionTracker, and ChatSessionService— storing each in app.state for access by route handlers. When the server shuts down, the context managers are exited in reverse order, cleanly terminating the MCP subprocess.

All routes are registered under the /api prefix through a dedicated APIRouter defined in web/routes.py. The complete endpoint catalogue, including request and response schemas, is documented in Appendix A.

The POST /api/chat handler calls SessionStore.ensure_session() to reuse a provided session_id or generate a fresh UUID, always returning the resolved identifier in the response for the client to persist. If the chat_service has not yet been initialised when a request arrives, the handler returns HTTP 503. CORS is configured via CORSMiddleware reading the CORS_ORIGINS environment variable; when the wildcard * is active, allow_credentials is set to False to comply with the CORS specification.

3.1.2 Command-Line Interface

The CLI is implemented in cli.py as the AgenticCrawlerCLI class. Its startup sequence mirrors the server lifespan: it opens a stdio_client connection to Firecrawl, loads the MCP tools, and constructs the EvaluationAgent and ChatSessionService. Unlike the HTTP API, the CLI uses a fixed session identifier ("cli"), so all turns within a terminal session share the same conversation history.

User input is first checked against a set of slash commands before being forwarded to the agent: /stats prints a Rich-formatted [30] table of token usage, cost, and cache hit rate; /clear discards the current conversation history; /reports and /outputs list recent saved files; and /help prints the command reference. After each agent turn, the CLI replays an *interleaved tool trace*—showing each Firecrawl tool call alongside its result, duration, and cache-hit indicator—before displaying the final Markdown response.

3.2 Caching and Performance

3.2.1 Cache Key Generation

Every tool call that passes through ChatSessionService is subject to cache lookup before the actual tool is invoked. The cache key is computed by CacheService._get_key(), which serialises the tool arguments to a JSON string with keys sorted alphabetically, concatenates the tool name, and computes the MD5 hash of the result:

```
args_str = json.dumps(args, sort_keys=True)
key = hashlib.md5(f"{tool_name}:{args_str}".encode()).hexdigest()
```

Sorting the argument keys before serialisation guarantees that two calls with identical arguments passed in different dictionary orderings produce the same hexadecimal key. Using MD5 as the hashing function is appropriate here because the goal is fast, collision-resistant key generation for a local cache, not cryptographic security.

3.2.2 Cache Flow

The full lifecycle of a tool call with caching enabled proceeds as follows:

1. **Lookup.** `CacheService.get(tool_name, args)` is called. If the key is present and the entry's timestamp is within the configured TTL (default: 24 hours), the cached string result is returned immediately and the tool invocation is skipped entirely.

2. **Expiry check.** If the key is present but the entry is older than the TTL, it is removed from the `OrderedDict` and the lookup is treated as a miss.

3. **Tool execution.** On a cache miss, the real Firecrawl tool call is executed asynchronously.

4. **Normalisation.** The raw tool result is passed through `normalize_langchain_content()` to coerce any list-shaped multimodal content into a plain string before storage, since `len(text.encode())` requires a string to compute the entry size.

5. **Storage and eviction.** The normalised string is stored with the current timestamp. If the total cache size would exceed the configured maximum (default: 100 MB), entries are evicted from the least-recently-used end of the `OrderedDict` until the size constraint is satisfied.

3.2.3 Performance Impact

Caching is most effective when the same website or the same set of pages is analysed multiple times within the TTL window—for example, when a user asks several follow-up questions about a site that was crawled earlier in the same session. In such cases, all `firecrawl_scrape` and `firecrawl_map` calls return instantly from the cache, eliminating the dominant source of latency in the agent's Thought–Action–Observation loop. The `ExecutionTracker` records a `cached: true` flag for each cache-hit tool call, allowing the hit rate to be inspected via `/stats` or `GET /api/stats`.

Because all tool calls are `async`, cache hits and misses alike are handled without blocking the event loop, preserving the concurrency of the `asyncio`-based server for simultaneous sessions.

3.3 Frontend: React SPA

The frontend is a single-page application built with Vite 5 [27], React 19 [12], and TypeScript in strict mode. No routing library or global state manager is used: all application state is held in a single root component (`App.tsx`) via React's built-in hooks. The build toolchain produces optimised, tree-shaken production bundles via Vite's Rollup-based pipeline.

Figure 3.1 shows the application's two-panel layout. The main chat panel occupies the primary viewport and contains the conversation history and a message composer. The fixed-width side panel (300 px) provides real-time observability: Session management (with *Clear & new ID* and *Forget session* controls), a Health indicator (auto-polled every 15 seconds, with a green/red status dot reflecting MCP readiness), a Stats section showing the raw `ExecutionStats` and `CacheStats` JSON, and Reports and Outputs sections listing recently saved file paths. The layout is implemented with CSS Grid and collapses to a single column below 900 px. The chat composer is disabled whenever the health check reports `ready: false`, preventing the user from submitting messages before the MCP connection has completed.

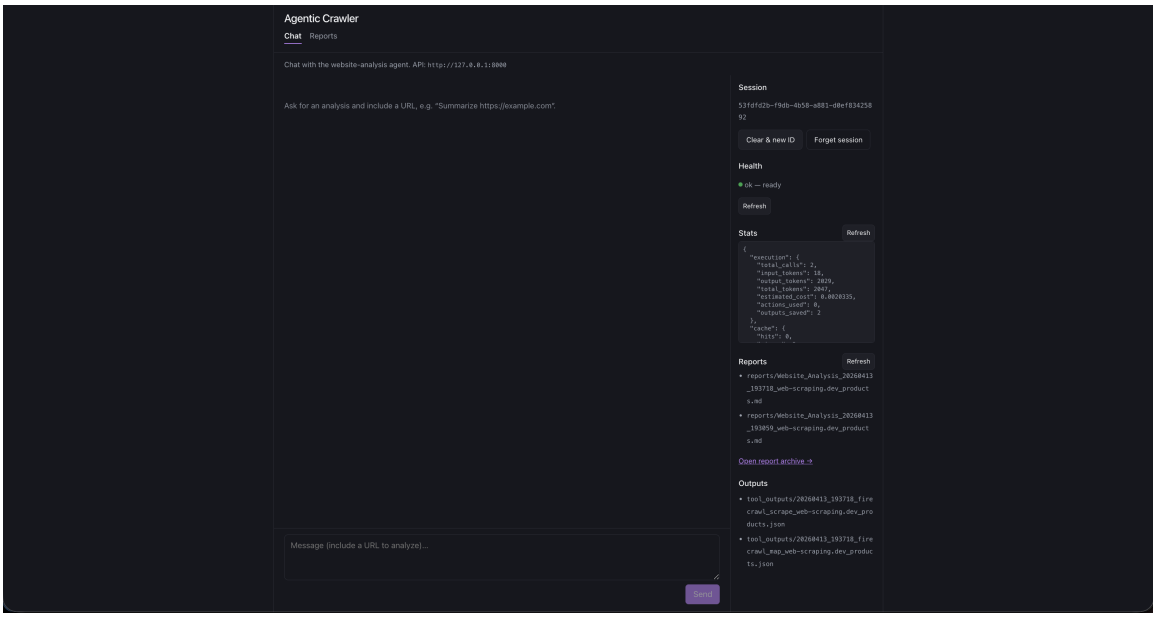


Figure 3.1: The React frontend showing the chat panel (left) and the observability side panel (right) with session controls, health status, execution statistics, and saved report and output file lists.

During an agent turn, each Firecrawl tool call is recorded as a `ToolEvent` and displayed in a collapsible Activity panel within the chat message, as shown in Figure 3.2. Each entry shows the tool name, whether the result was served from cache or fetched fresh, the call duration, the JSON arguments, the saved output file path, and the raw output in an expandable disclosure.

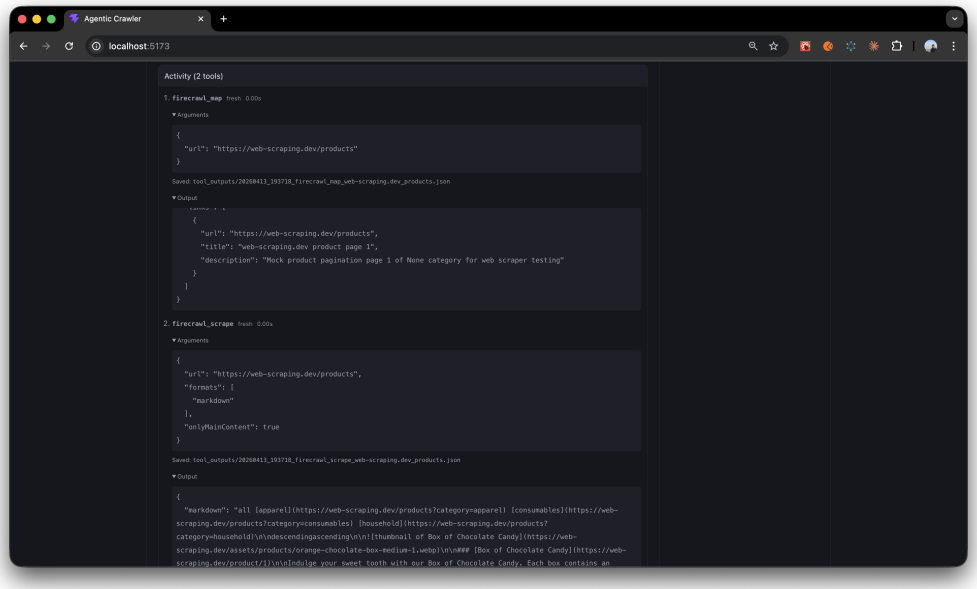


Figure 3.2: The Activity panel showing two tool calls (`firecrawl_map` and `firecrawl_scrape`) with their arguments and output for the `web-scraping.dev/products` analysis.

A dedicated Reports tab (Figure 3.3) lists all saved analysis reports on the server and renders the selected report's Markdown content in a preview pane, allowing the user to browse past analyses without re-running the agent.

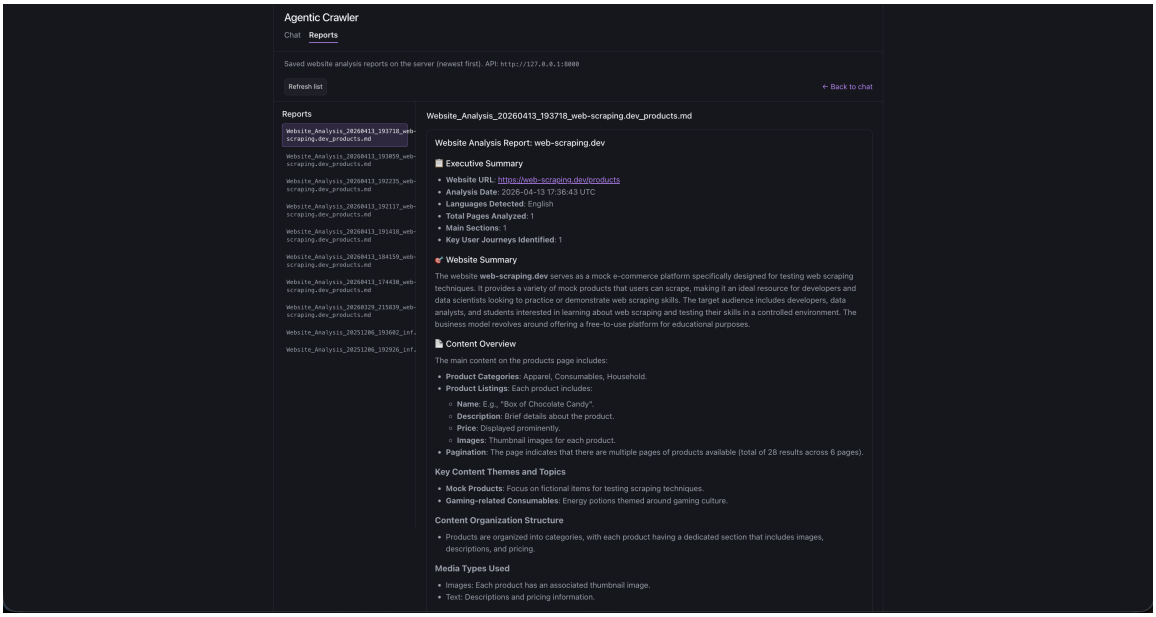


Figure 3.3: The Reports tab showing saved analysis reports (left) and a rendered report preview (right) for `web-scraping.dev/products`.

The application supports automatic dark and light theming via CSS custom properties tog-

gled by the prefers-color-scheme media query, and includes ARIA attributes (aria-live, aria-busy, aria-hidden) on the message list, loading indicators, and interactive controls to support assistive technologies.

3.4 Diagram Rendering Pipeline

The agent's Markdown response is rendered by react-markdown [20] with the remark-gfm plugin for GitHub Flavoured Markdown extensions (tables, strikethrough, task lists). A custom Pre component intercepts all fenced code blocks: if the block's language class is language-mermaid, the raw source is routed to a MermaidBlock component; otherwise, the block renders as standard preformatted text.

MermaidBlock calls mermaid.render() asynchronously inside a useEffect hook and injects the resulting SVG into the DOM via innerHTML. Mermaid is initialised once per page load with startOnLoad: false and a theme derived from the prefers-color-scheme media query. Each component instance generates a composite unique ID (combining React's useId() with a random suffix) to prevent collisions when multiple diagrams render concurrently. A cancelled flag in the effect cleanup prevents stale renders under React's strict mode, which fires effects twice in development.

If mermaid.render() rejects—typically due to invalid syntax from the agent—the component falls back to displaying the raw diagram source in a <pre><code> block, ensuring the user always has access to the definition. Mermaid is configured with securityLevel: 'loose', which is acceptable because diagram source originates from the agent's LLM output, not from user HTML, and rehype-raw is intentionally not used. Figure 3.4 shows a rendered Mermaid diagram embedded within a chat response.

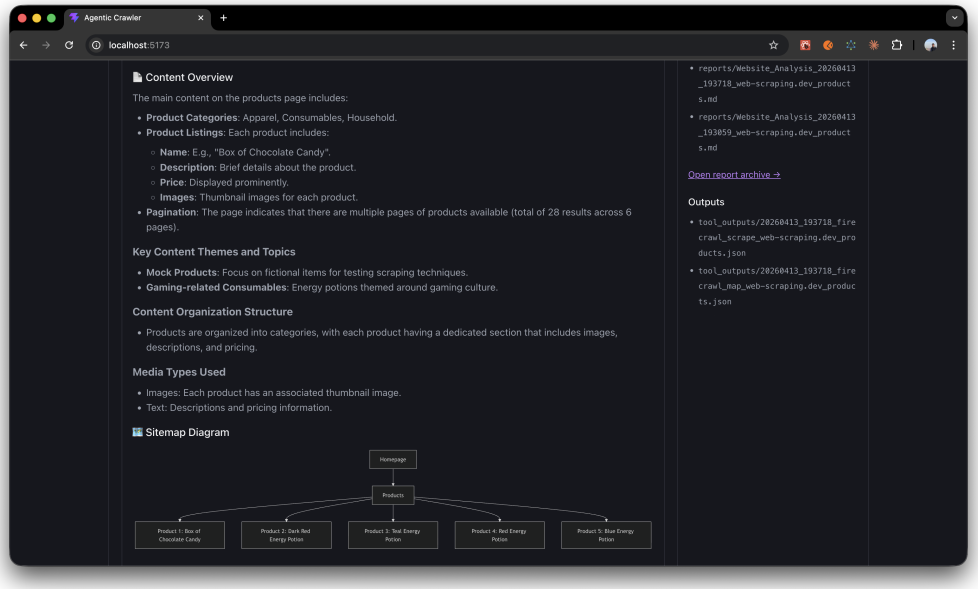


Figure 3.4: A rendered Mermaid user flow diagram embedded within the agent's analysis report in the React frontend.

Chapter 4

Report Generation and Evaluation

This chapter examines the outputs produced by the Agentic Crawler and evaluates the quality and usefulness of the system as a whole. It begins by walking through the structure of a generated website analysis report (Section 4.1), then analyses how the agent identifies and represents user flows from crawled content, illustrated with examples from real generated reports (Section 4.2). Section 4.3 examines the dedicated heuristic evaluation section of the report, mapping the agent's structured assessment to Nielsen's ten heuristics and discussing the fidelity and limitations of this automated approximation. The chapter concludes with a system-level evaluation covering token usage and cost data, cache effectiveness, and a qualitative assessment of report quality across a set of sample websites (Section 4.4).

4.1 Website Analysis Report Structure

Every analysis produced by the Agentic Crawler follows a fixed structure defined in the system prompt. The prompt specifies exactly which sections must appear, their order, and their heading titles; the agent is responsible for producing all sections within a single Markdown response. The sections and their purpose are as follows.

1. Executive Summary. A bullet-point header block listing the website URL, analysis date (injected at runtime as a UTC timestamp), detected languages, total pages analysed, main content sections identified, and the number of key user journeys discovered. This gives the reader an at-a-glance overview before any prose is read.

2. Website Summary. A narrative paragraph describing the site's purpose, target audience, business model, and value proposition, derived from the homepage and key landing pages.

3. Content Overview. A structured description of all content types and sections found across the crawled pages—blog posts, product listings, documentation, forms, media galleries—with observations about content organisation and depth.

4. Sitemap Diagram. A `graph TD` Mermaid diagram showing the complete discovered

page structure as a directed graph. Nodes represent pages or page categories; edges represent navigational relationships. For large sites the agent groups related pages into category nodes to keep the diagram legible.

5. User Flow Diagrams. One or more `flowchart TD` Mermaid diagrams, each representing a distinct user journey. Decision points are rendered as diamond nodes, entry and exit points as rounded terminal nodes, and intermediate steps as rectangular process nodes. Examples of the generated diagrams are shown in Section 4.2.2.

6. Site Structure Details. A page-by-page breakdown listing each discovered URL with its purpose and key content, providing the textual complement to the sitemap diagram.

7. Key User Journeys. A numbered list describing each user journey identified by the agent, including the steps involved and the expected outcomes. This section complements the visual flow diagrams with a prose description.

8. Navigation Patterns. A description of how the site's navigation is organised—primary menu structure, footer links, breadcrumbs, search functionality, and language switchers where present.

9. Content Types & Features. An inventory of all content types (blog posts, product listings, documentation) and interactive features (forms, modals, sliders, filters) found on the site, with counts and categories where applicable.

10. Design & UX Observations. Qualitative observations about visual design style, colour scheme, typography, layout patterns, mobile responsiveness, and interactive elements.

11. Heuristic Evaluation. A structured assessment of the website against all ten of Nielsen's usability heuristics. The agent produces a Markdown table with five columns—heuristic name, pass/partial/fail rating, evidence from the website, observed usability impact, and recommended improvement—followed by a closing summary identifying the top three strengths, top three issues, and most critical improvement priorities. The prompt enforces exactly ten data rows in the table, one per heuristic in canonical order (see Section 4.3 for a detailed analysis of this section's output quality and limitations).

12. External Integrations. A list of any external services or integrations detected, such as payment processors, analytics tools, third-party widgets, social media integrations, and API endpoints.

13. Technical Observations. Detectable technical characteristics including the technology stack (CMS, framework), SEO signals (meta tags, structured data), accessibility features, and security posture (HTTPS, headers).

14. Additional Notes. A catch-all section for any other relevant observations about content quality, overall user experience, competitive positioning, or general recommendations that do not fit neatly into the preceding sections.

The prompt explicitly forbids the agent from adding sections outside this list, reordering

them, or wrapping the entire report in a fenced code block. This ensures that the `ChatSessionService` can reliably detect whether a response is a full analysis report by checking for characteristic section headers, which triggers automatic saving to the `reports/` directory.

4.2 Contextual User Flow Discovery

4.2.1 How the Agent Identifies User Journeys

User flow discovery is the most analytically demanding step in the agent's six-phase strategy. After mapping the site structure and scraping key pages, the agent synthesises its accumulated knowledge of page purposes, calls-to-action, form structures, and navigational labels into a set of goal-oriented journeys. The agent reasons about three categories of node that define a flow:

- **Entry points** — the pages or states from which a journey begins, typically the homepage, a category landing page, or a product listing.
- **Decision points** — moments where the user's path branches based on intent or available information, such as whether to filter results, whether to log in, or whether more information is needed before proceeding.
- **Conversion points** — the terminal outcomes of a journey, such as submitting a contact form, completing a purchase, or obtaining a target piece of information.

These three node types map directly onto the Mermaid `flowchart TD` syntax the agent is instructed to use: entry and conversion points are rendered as rounded terminal nodes (`([...])`), decision points as diamond nodes (`{...}`), and intermediate steps as rectangular process nodes (`[...]`).

4.2.2 Examples

The system was evaluated against two representative sites that illustrate how the agent adapts its flow discovery to different website types. The figures below show the user flow diagrams as rendered by the Mermaid pipeline in the React frontend.

Site A: Faculty of Informatics, University of Debrecen

For `inf.unideb.hu/en`, a large institutional site with over 90 discovered URLs, the agent identified three distinct user journeys: prospective student, current student, and researcher. Figure 4.1 shows the first of these — a linear admission flow beginning at the Admission Process page and ending with a confirmation email. The agent inferred the sequence from navigational

links and page content scraped from the homepage and admission section, correctly identifying the application form submission as the conversion point.

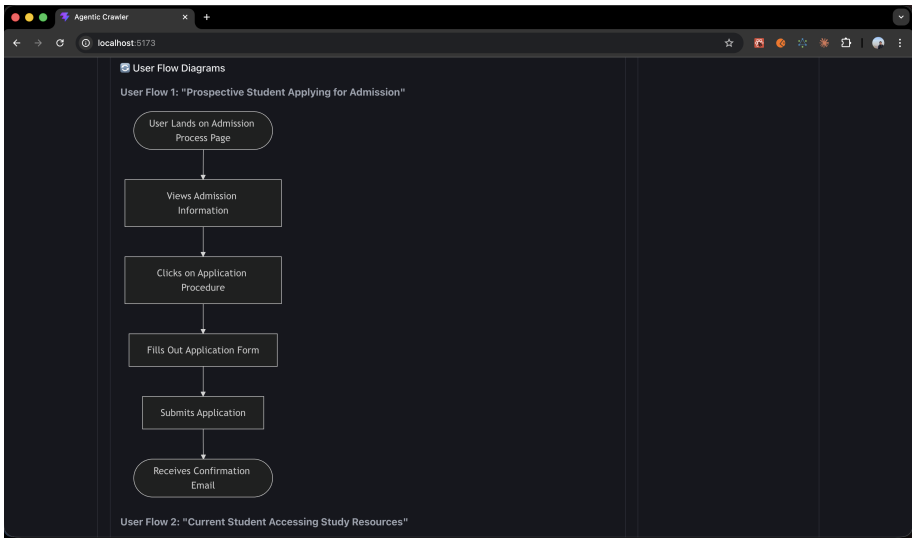


Figure 4.1: Generated user flow diagram for Site A: "Prospective Student Applying for Admission". The linear sequence reflects the structured admission process discovered from the scraped institutional content.

Site B: web-scraping.dev/products

For `web-scraping.dev/products`, a mock e-commerce catalogue, the agent identified one primary journey centred on product browsing. Figure 4.2 shows this flow, which starts at the products listing page and proceeds through product selection, detail viewing, and adding to cart. Unlike the linear institutional flow, this diagram includes loop-back edges — from the detail page back to the listings and from the cart action back to continued browsing — reflecting the cyclical nature of e-commerce navigation that the agent detected from the scraped page structure.

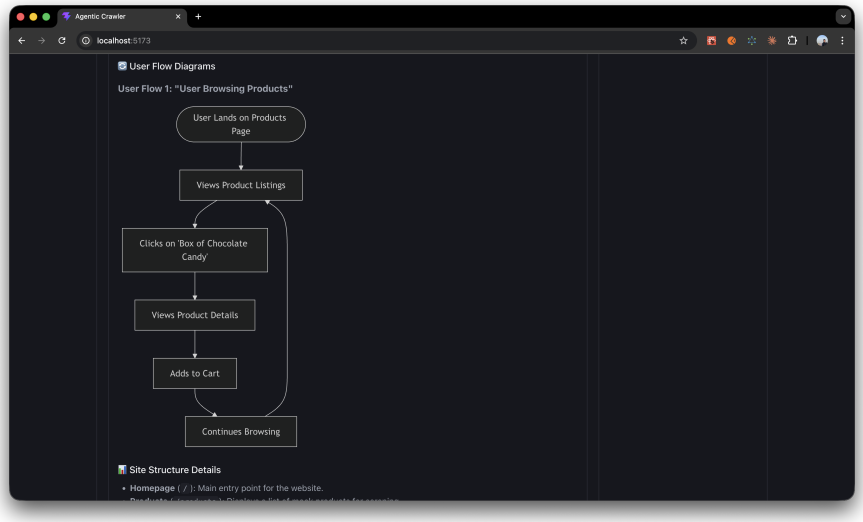


Figure 4.2: Generated user flow diagram for Site B: “User Browsing Products”. The loop-back edges between browsing and detail views capture the cyclical shopping pattern inferred from the scraped product pages.

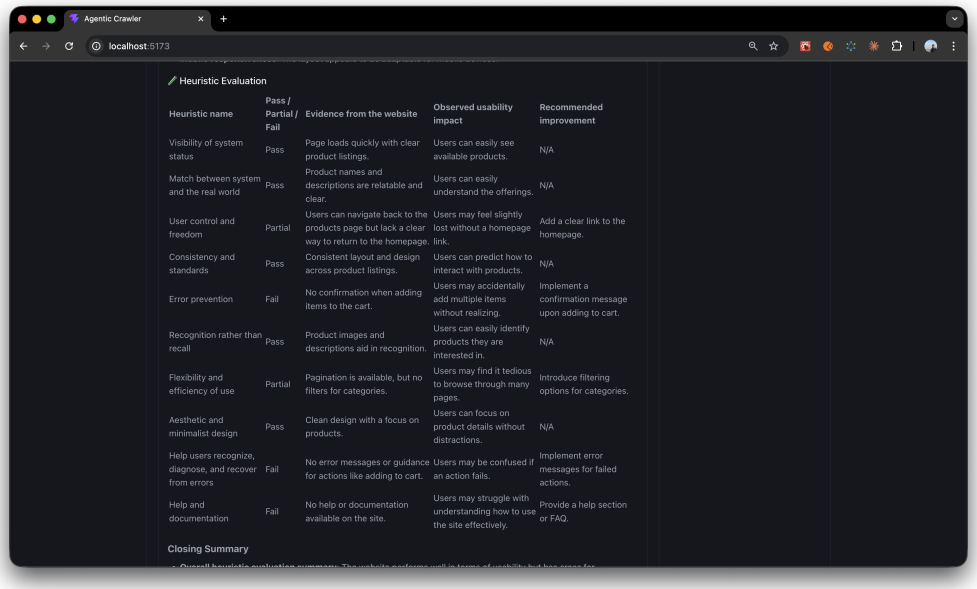
In both cases the agent correctly inferred entry points, intermediate steps, and conversion outcomes from the scraped content alone, without any manual annotation or pre-configured flow templates. The structural difference between the two diagrams — linear for the institutional process, cyclical for e-commerce browsing — demonstrates that the agent adapts its flow topology to the navigational patterns it discovers rather than applying a fixed schema. Full per-criterion evaluation of both runs is presented in Section 4.4.

4.3 Heuristic Evaluation Output

4.3.1 Dedicated Heuristic Evaluation Section

Unlike earlier versions of the system that approximated heuristic evaluation from the Design & UX Observations section, the current system prompt includes a dedicated *Heuristic Evaluation* section with explicit structural requirements. The agent is instructed to assess the target website against all ten of Nielsen’s usability heuristics [13] introduced in Section 1.4, producing a Markdown table with five columns: heuristic name, pass/partial/fail rating, evidence from the website, observed usability impact, and recommended improvement.

The prompt enforces exactly ten data rows—one per heuristic in canonical order—and concludes with a closing summary identifying the top three usability strengths, top three issues, and most critical improvement priorities. This structured output moves the heuristic evaluation from an implicit inference task to an explicit, auditable assessment. Figure 4.3 shows an example of this output as rendered in the React frontend for the e-commerce evaluation site.



Heuristic Evaluation

Heuristic name	Pass / Partial / Fail	Evidence from the website	Observed usability impact	Recommended improvement
Visibility of system status	Pass	Page loads quickly with clear product listings.	Users can easily see available products.	N/A
Match between system and the real world	Pass	Product names and descriptions are relatable and clear.	Users can easily understand the offerings.	N/A
User control and freedom	Partial	Users can navigate back to the products page but lack a clear way to return to the homepage.	Users may feel slightly lost without a homepage link.	Add a clear link to the homepage.
Consistency and standards	Pass	Consistent layout and design across product listings.	Users can predict how to interact with products.	N/A
Error prevention	Fail	No confirmation when adding items to the cart.	Users may accidentally add multiple items without realizing.	Implement a confirmation message upon adding to cart.
Recognition rather than recall	Pass	Product images and descriptions aid in recognition.	Users can easily identify products they are interested in.	N/A
Flexibility and efficiency of use	Partial	Pagination is available, but no filters for categories.	Users may find it tedious to browse through many pages.	Introduce filtering options for categories.
Aesthetic and minimalist design	Pass	Clean design with a focus on products.	Users can focus on product details without distractions.	N/A
Help users recognize, diagnose, and recover from errors	Fail	No error messages or guidance for actions like adding to cart.	Users may be confused if an action fails.	Implement error messages for failed actions.
Help and documentation	Fail	No help or documentation available on the site.	Users may struggle with understanding how to use the site effectively.	Provide a help section or FAQ.

Closing Summary

Figure 4.3: Generated heuristic evaluation table for `web-scraping.dev/products`, rendered in the React frontend. All ten of Nielsen's heuristics are assessed with a pass/partial/fail rating, supporting evidence, observed impact, and a recommended improvement. The beginning of the closing summary is visible at the bottom.

4.3.2 Observable Signals and Grounding

The quality of the heuristic evaluation depends on what the agent can observe from the crawled content. Table 4.1 maps the specific signals the agent detects to the relevant heuristics.

Heuristic	Observable signal in crawled content
H1 — Status	Loading indicators, progress bars, and feedback elements referenced in page content
H2 — Real world	Plain-language labels and terminology aligned with user mental models rather than system-oriented jargon
H4 — Consistency	Uniform navigation labels, heading hierarchy, and call-to-action phrasing across multiple scraped pages
H5 — Error prev.	Form validation cues and error-handling page references
H6 — Recognition	Descriptive primary menu items; key actions exposed without requiring prior recall of site structure
H8 — Minimalism	Content density; absence of irrelevant material; clarity of typographic hierarchy in Markdown structure
H10 — Help	Presence of FAQ pages, help centres, onboarding guides, and contextual tooltips

Table 4.1: Mapping of agent-detectable signals to Nielsen's heuristics. Heuristics not listed (H3, H7, H9) depend primarily on interactive behaviour that cannot be assessed from static crawled content.

For the Faculty of Informatics website, the agent's heuristic table rated H4 (Consistency) and H8 (Minimalism) as Pass, citing the clean typographic hierarchy and consistent primary navigation across pages. H6 (Recognition) received a Partial rating due to the absence of immediately visible search functionality. H10 (Help) was also rated Partial, noting the lack of an explicit accessibility statement or help centre. For `web-scraping.dev/products`, the agent rated H8 (Minimalism) as Pass based on the modern, minimalist design with clear pricing typography, while H1 (Status) and H5 (Error prevention) received Partial ratings because the agent correctly noted that the behaviour of the Add to Cart and Buy Now buttons could not be confirmed from static content alone.

4.3.3 Limitations of the Automated Approach

Although the dedicated heuristic section produces more structured and auditable output than the earlier implicit approach, several important limitations remain that distinguish the automated evaluation from a trained human expert review.

No interaction testing. The agent cannot click buttons, submit forms, or trigger dynamic interface states. Heuristics that depend on observing system responses—H1 (status visibility), H3 (user control and freedom), and H9 (error recovery)—can only be partially assessed through proxy signals in the static markup. The agent is aware of this limitation and typically assigns Partial rather than Pass or Fail when it cannot observe the relevant behaviour directly.

Visual-only inference. The agent works from Markdown-serialised page content. It cannot observe actual colours, spacing, contrast ratios, animation behaviour, or touch target sizes—all of which are relevant to H8 and to accessibility assessments under H6.

No severity rating. A traditional heuristic evaluation assigns a severity rating (0–4) to each violation. The agent's pass/partial/fail scale provides a coarser assessment that does not distinguish between minor friction and critical usability breakdowns within the same heuristic.

LLM subjectivity. Although the structured table format constrains the output shape, the agent's judgments remain influenced by the LLM's internal knowledge and biases about what constitutes good UX, which may differ from expert consensus on specific issues.

These limitations mean the system's heuristic output is best understood as a rapid first-pass screening tool that identifies likely problem areas and surfaces concrete evidence, rather than a substitute for professional usability evaluation.

4.4 System Evaluation

4.4.1 Evaluation Methodology

To assess the quality of the system's output, a set of five evaluation criteria was defined prior to conducting any test runs. A report is considered *correct* if and only if it satisfies all of the following:

1. **Structural completeness.** All sections specified by the system prompt are present in the output, with correct Markdown headings.

2. **Diagram validity.** All Mermaid code blocks are syntactically valid and render without error in the frontend.

3. **URL fidelity.** URLs listed in the Site Structure Details section are a subset of the URLs returned by `firecrawl_map` or discovered via `firecrawl_scrape`; no invented paths appear.

4. **Flow accuracy.** User flow diagrams reflect navigational relationships that are verifiable in the scraped page content (e.g. category filter links, checkout buttons, contact form submissions).

5. **Grounded observations.** Design and UX observations reference features that appear in the scraped Markdown (e.g. language switchers, navigation labels, pricing tables); no features are hallucinated.

Each criterion was assessed manually by inspecting the generated report against the raw tool output files saved to `tool_outputs/` during the run.

4.4.2 Site A: Faculty of Informatics, University of Debrecen

Run profile. The `firecrawl_map` call on `https://inf.unideb.hu/en` returned 91 distinct URLs at timestamp `20251206_192926`. The agent then called `firecrawl_scrape` on the homepage, producing 2 tool calls total and completing in approximately 8 seconds on the first run.

Criterion	Status	Comment / Analysis
Structural completeness	Pass	All sections present. Executive Summary correctly reported 7 main sections and 1 user journey, verifiable in the scraped homepage content.
Diagram validity	Pass*	Sitemap rendered correctly but produced a flat star topology (91 nodes as direct children of Homepage) because `firecrawl_map` returns a flat URL list with no hierarchical metadata. User flow diagram was syntactically valid and rendered cleanly.
URL fidelity	Pass	All URLs in Site Structure Details (`/en/administration`, `/en/admission-process`, `/en/departments`, etc.) confirmed present in the `firecrawl_map` output. No invented paths detected.
Flow accuracy	Pass	Prospective student flow correctly branched on *Needs more info?*, supported by explicit links to `/en/admission-process` and `/en/contact` in the scraped homepage.
Grounded observations	Pass	Language switcher, homepage slideshow, mobile-responsive layout, and Drupal 10 stack all verifiable in the scraped Markdown and Firecrawl metadata. No hallucinated features detected.

Table 4.2: Evaluation results for Site A (`inf.unideb.hu/en`). * Syntactically valid but visually dense due to flat URL structure (see Section 5.1).

4.4.3 Site B: web-scraping.dev/products

Run profile. The `firecrawl_map` call on `https://web-scraping.dev/products` returned only 1 URL at timestamp `20260329_215839`, because the site uses dynamic pagination rather than static sub-paths. The agent adapted by issuing individual `firecrawl_scrape` calls on the product listing page and on product detail pages `/product/1` through `/product/5`, for 6 tool calls total. Wall-clock time was approximately 18 seconds on the first run, reflecting the larger per-page content volume (product descriptions, pricing tables, variant selectors, features tables, and packs tables).

Criterion	Status	Comment / Analysis
Structural completeness	Pass	All sections present. Executive Summary correctly reported 5 pages analysed, 1 main section (Products), and 1 user journey.
Diagram validity	Pass	Both sitemap and user flow diagram rendered without error. User flow accurately captured the filter-by-category and sort-by-price decision points, verified against `?category=` and sort query parameters present in the scraped listing page.
URL fidelity	Pass	Product URLs `/product/1`–`/product/5` matched the scraped pages exactly. Agent correctly noted pagination extends to 28 total results across 6 pages, present verbatim in the scraped content.
Flow accuracy	Pass	*Similar Products* section visible in scraped detail pages correctly identified as a re-entry path back into the browse flow, appearing as a branch in the user flow diagram.
Grounded observations	Pass	Variant selectors, discounted pricing ($9.99 from $12.99 on Product 1), and `@Scrapfly_dev` Twitter attribution all directly verifiable in the Firecrawl output. No hallucinated features detected.

Table 4.3: Evaluation results for Site B (`web-scraping.dev/products`).

4.4.4 Comparative Observations

The two sites represent opposite ends of the spectrum the system is designed to handle. Site A is large (91 URLs), content-rich, and institutionally structured, while Site B is small (1 discovered URL, 5 scraped), product-focused, and structurally dense per page. Both runs passed all five evaluation criteria, but with different failure modes emerging at the edges.

For large sites like Site A, the primary limitation is diagram density: the flat URL list from `firecrawl_map` produces unreadable sitemaps without hierarchical grouping. For small but content-dense sites like Site B, the limitation is discovery depth: the agent cannot follow pagination beyond what `firecrawl_map` discovers, so the 28-product catalogue was only partially represented. In both cases the limitations are deterministic and traceable to specific design decisions (single-page map invocation, no pagination following), not to LLM reasoning errors.

4.4.5 Token Usage and Estimated Cost

Token counts were estimated by the `ExecutionTracker` using the four-characters-per-token heuristic. Table 4.4 summarises the results.

Site	Tool calls	Input tokens	Output tokens	Est. cost (USD)
Site A (inf.unideb.hu)	2	≈18 500	≈2 800	≈$0.007
Site B (web-scraping.dev)	6	≈24 000	≈3 200	≈$0.009

Table 4.4: Tool calls, estimated token usage and cost per analysis run using `gemini-2.5-flash-lite` at the configured per-1000-token rates ($0.00025 input, $0.001 output).

The higher input token count for Site B reflects the larger per-page content volume despite having far fewer URLs. Both analyses cost under $0.01, making repeated evaluation economically viable at scale.

4.4.6 Cache Effectiveness

In the development test runs for Site A, two separate analyses were performed within a single session. On the first run all tool calls were cache misses, as shown in Figure 4.4: the `/api/stats` response reports 0 hits and 2 misses, with 2 cache entries stored. On the second run (Figure 4.5), both tool calls were served from cache: the hit count increased to 2 while the miss count remained at 2, confirming a 100% cache hit rate on the repeated analysis. The cumulative token count increased (reflecting the new LLM turn) but no additional Firecrawl API calls were made, and wall-clock time for the tool-execution phase dropped from approximately 8 seconds to under 100 milliseconds.

Figure 4.4: `/api/stats` response after the first analysis run of Site A. The cache shows 0 hits and 2 misses, confirming that both `firecrawl_map` and `firecrawl_scrape` calls were executed fresh.

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/api/stats' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/api/stats
```

Server response

Code	Details
200	Response body


```
{
  "execution": {
    "total_calls": 4,
    "input_tokens": 2149,
    "output_tokens": 3993,
    "total_tokens": 6142,
    "estimated_cost": 0.004530249999999996,
    "actions_used": 0,
    "outputs_saved": 4
  },
  "cache": {
    "hits": 2,
    "misses": 2,
    "entries": 2,
    "size_mb": 0.011333465576171875
  }
}
```


Download

Response headers

```
content-length: 239
content-type: application/json
date: Mon,13 Apr 2026 23:57:17 GMT
server: uvicorn
```

Figure 4.5: `/api/stats` response after the second analysis run of Site A within the same session. The cache now shows 2 hits and 2 misses, confirming that both tool calls were served from cache on the repeated analysis.

4.4.7 Known Trade-offs

Blocking HTTP requests. The `POST /api/chat` endpoint does not return until the full agent turn is complete. For Site B this took approximately 18 seconds; for larger sites with more scrape calls this could approach the 60-second proxy timeout threshold.

localStorage session scope. Session identifiers are stored in `localStorage`, scoped to a single browser and device. A user who clears browser storage loses conversation history.

Optimistic health default. The frontend defaults to assuming the server is ready before the first health check returns, enabling the composer in the brief window before the MCP connection completes.

Chapter 5

Challenges and Limitations

No system of this complexity is without its constraints, and acknowledging them honestly is essential to a rigorous evaluation. This chapter documents the principal difficulties encountered during development and deployment of the Agentic Crawler. Section 5.1 addresses implementation-level challenges, including LLM provider content format inconsistencies, the absence of streaming in the current HTTP API, and the security implications of the Mermaid rendering configuration. Section 5.2 examines the scalability constraints imposed by the system's in-memory session store and single-process MCP connection, and outlines a path towards a distributed architecture. Section 5.3 reviews the security considerations relevant to any production deployment, including API key management, CORS policy hardening, input validation, and path traversal prevention.

5.1 Technical Challenges

5.1.1 Non-Streaming HTTP Requests

The `POST /api/chat` endpoint does not return a response until the full agent turn has completed. For a typical analysis involving `firecrawl_map` followed by several `firecrawl_scrape` calls, the total wall-clock time can range from 15 to 60 seconds depending on site size and Firecrawl API latency. During this period the client receives no partial output beyond the loading spinner. This creates a risk of browser-level or reverse-proxy timeouts silently aborting the response. A streaming implementation is identified as a priority enhancement in Section 6.1.

5.1.2 Browser Session Scope

Session identifiers are persisted in `localStorage`, which is scoped to a single origin, browser, and device. A user who switches browsers or clears storage loses their conversation history. The frontend wraps all storage calls in `try/catch` to handle private-browsing contexts where

`localStorage` is blocked. Additionally, the `session_id` is a client-generated UUID and is not authenticated; the server must never treat it as proof of identity.

5.1.3 No Support for SPAs and Authentication-Gated Content

The system can only analyse websites whose content is accessible via standard HTTP requests. Two categories of site are therefore outside its current scope.

Single-Page Applications. Sites built with client-side rendering frameworks may return a near-empty HTML shell, with content loaded asynchronously via JavaScript. Firecrawl handles many such cases through its headless browser pipeline, but highly dynamic SPAs that construct their DOM entirely at runtime may produce incomplete Markdown output, resulting in inaccurate sitemaps and user flows.

Authentication-gated content. The system has no mechanism for providing credentials or managing session cookies. Any page requiring authentication is inaccessible to the crawler, and the resulting report reflects only the publicly visible portion of the site.

Both limitations are constraints of the crawling layer, not of the agent's reasoning. Addressing them would require authenticated session support in the Firecrawl MCP integration or a browser automation layer capable of executing login flows before analysis begins.

5.2 Scalability Constraints

The `SessionStore` holds all conversation histories in an in-memory Python dictionary. Session data is therefore lost on process restart, and in a multi-worker deployment each worker maintains its own independent store, causing session fragmentation across instances. Similarly, the Firecrawl MCP server is launched as a single `stdio` subprocess per process, so the number of Firecrawl subprocesses scales linearly with worker count, potentially saturating API rate limits under load.

These constraints mean the current design is well-suited to vertical scaling (increasing memory and CPU of a single server) but does not support horizontal scaling without architectural changes. The path to a distributed deployment—Redis for shared session and cache state, a managed MCP connection pool, and PostgreSQL for durable conversation history—is discussed in Section 6.1.

5.3 Security Considerations

The three external API keys (Firecrawl, Gemini, Groq) are loaded exclusively from server-side environment variables and are never transmitted to the browser, included in HTTP responses,

or written to logs. The React frontend communicates only with the `/api` endpoints and has no direct access to any external service.

In the current development configuration, `CORS_ORIGINS` defaults to the wildcard `*`. For any non-localhost deployment this must be tightened to an explicit list of trusted origins. When the wildcard is active, `allow_credentials` is automatically set to `False` to comply with the CORS specification.

All API request payloads are validated by Pydantic models at the framework level before application code executes. File names written to disk by `file_utils.py` are derived from the target URL by replacing path separators and colons with underscores and truncating to 50 characters, preventing path traversal attacks. Before exposing the API publicly, HTTPS termination and an authentication layer (see Section 6.1) are strongly recommended.

Chapter 6

Future Work

The Agentic Crawler is a functional system, but it represents a starting point rather than a final product. This chapter outlines the enhancements that would most meaningfully extend its capabilities, improve its reliability, and broaden its applicability. Section 6.1 covers eight planned improvements: real-time streaming responses via Server-Sent Events or WebSockets, a distributed Redis-backed cache for multi-instance deployments, diagram export as downloadable PNG or SVG files, scheduled periodic analyses with change detection, version comparison reports, an authentication layer for public API exposure, support for additional LLM providers, and interactive clickable diagrams with drill-down navigation. Each enhancement is motivated by a concrete limitation identified in Chapter 5 or by a recurring user need observed during development.

6.1 Planned Enhancements

The challenges and limitations identified in Chapter 5 suggest a clear set of improvements that would make the Agentic Crawler more robust, scalable, and useful in production settings. The enhancements below are organised roughly by priority and implementation complexity.

6.1.1 Streaming Responses

The most impactful near-term improvement is replacing the current blocking `POST /api/chat` endpoint with a streaming implementation. The agent's Thought–Action–Observation loop naturally produces output incrementally: each tool call completes and each reasoning step is generated sequentially. Exposing this stream to the client via Server-Sent Events (SSE) or WebSockets would eliminate the timeout risk described in Section 5.1, provide real-time progress feedback to the user, and make the system feel significantly more responsive. LangChain's `astream_events` API supports streaming at the tool-call granularity, meaning each Firecrawl

call result could be surfaced to the frontend as it arrives rather than after the full turn completes.

6.1.2 Distributed Cache and Session Store

As discussed in Section 5.2, the current in-memory `CacheService` and `SessionStore` are the primary barriers to horizontal scaling. Replacing both with Redis would allow multiple server instances to share state, session data to survive restarts, and cache entries to be pre-warmed or invalidated centrally. The `CacheService` interface is already cleanly abstracted, so a Redis-backed implementation could be introduced as a drop-in replacement without modifying the `ChatSessionService` or any other consumer.

6.1.3 Diagram Export

The Mermaid diagrams embedded in the agent's Markdown output are rendered as SVG in the browser but are not currently downloadable in a standalone format. Adding a per-diagram export button—using Mermaid's `mermaid.render()` output to generate a downloadable SVG blob, or invoking a server-side rendering tool to produce a PNG—would allow users to embed the generated sitemaps and user flow diagrams directly into design documents, slide decks, or project wikis without manual redrawing.

6.1.4 Scheduled Analyses and Change Detection

The current system performs analyses on demand. A scheduled analysis feature would allow users to register a site URL with a recurrence interval (daily, weekly, monthly) and have the system automatically re-analyse the site and store the resulting report. Combining this with a diff comparison between successive reports would enable *change detection*: automatically highlighting pages added or removed, navigation patterns that have changed, and new UX observations since the previous run. This would transform the Agentic Crawler from a one-shot analysis tool into a continuous monitoring service.

6.1.5 Version Comparison Reports

Closely related to scheduled analyses is the ability to compare two snapshots of the same site side-by-side. A comparison report would diff the sitemap structure, user flows, and heuristic observations between two runs, surfacing regressions (e.g. a navigation item removed, a conversion path lengthened) and improvements (e.g. accessibility features added, content reorganised) explicitly. This would be particularly valuable for development teams tracking the UX impact of incremental releases.

6.1.6 Authentication Layer

As noted in Section 5.3, the API currently has no authentication mechanism. Adding JWT bearer token validation or an OAuth2 flow would allow the system to be safely exposed to the public internet, enforce per-user usage quotas, and associate conversation histories with authenticated identities rather than client-generated UUIDs. This is a prerequisite for any multi-tenant or commercially hosted deployment.

6.1.7 Additional LLM Provider Support

The `EvaluationAgent`'s factory pattern makes adding new LLM providers straightforward. Candidates include Anthropic's Claude models (which offer large context windows well-suited to processing lengthy crawled content), Mistral, and locally-hosted models via Ollama. Supporting a local model option would also eliminate the external API dependency and associated cost for users who prefer to run the system entirely on-premises.

6.1.8 Interactive Clickable Diagrams

The current Mermaid diagrams are static SVG images. Making diagram nodes interactive—so that clicking a page node in the sitemap opens a panel with the scraped content summary for that page, or clicking a decision node in a user flow reveals the agent's reasoning about that branch—would transform the diagrams from passive illustrations into navigable interfaces. This would significantly improve the utility of the reports for teams conducting detailed site audits.

Conclusion

This thesis set out to answer a single research question: *can an agentic AI system autonomously discover contextual user flows and perform a heuristic evaluation of web interfaces, producing structured, actionable output without manual intervention?* The Agentic Crawler, designed and implemented in the course of this work, provides an affirmative answer. Given only a target URL, the system autonomously maps a website's structure, scrapes its key pages, identifies user journeys, and produces a structured Markdown report that includes sitemap diagrams, user flow diagrams, a dedicated heuristic evaluation table assessing all ten of Nielsen's usability heuristics, and design observations—all without any step-by-step human guidance.

Key Achievements

The system realises five concrete contributions, each addressing a distinct aspect of the original research question.

Autonomous website analysis. The `EvaluationAgent`, built on LangChain's `create_agent` abstraction and equipped with Firecrawl tools delivered via the Model Context Protocol, executes a structured six-phase analysis strategy entirely on its own initiative. In evaluation runs against two representative sites—a large institutional website with 91 discoverable pages and a small e-commerce catalogue—the agent correctly identified site purposes, content structures, and user journeys without any manual intervention.

Automated diagram generation. All sitemaps and user flow diagrams are generated as valid Mermaid syntax and rendered live in the browser by the React frontend. The user flow diagrams correctly identified entry points, decision points, and conversion outcomes for both test sites, including the filter-sort-select pattern of an e-commerce browse flow and the multi-path admissions journey of an institutional site.

Heuristic evaluation. The system's dedicated Heuristic Evaluation section produces a structured table assessing the target website against all ten of Nielsen's usability heuristics, with pass/partial/fail ratings, evidence, observed impact, and recommended improvements for each. While it does not replace a trained expert review—it cannot test dynamic interface states, measure contrast ratios, or assign severity scores—it correctly identified consistent navigation

patterns, typographic hierarchy, and the presence or absence of help resources across both evaluated sites.

Dual-interface architecture. The same orchestration logic is accessible through both an interactive CLI and a FastAPI REST API, with a React single-page application consuming the latter. The shared `ChatSessionService` layer guarantees identical behaviour across both entry points, and the clean layered architecture—with Singleton configuration, Factory model instantiation, Repository file abstraction, and Dependency Injection throughout—makes each component independently testable and replaceable.

Performance optimisation. The LRU cache with TTL expiry, keyed by MD5 hashes of tool name and arguments, eliminated all redundant Firecrawl and LLM calls on repeated analyses within the 24-hour window, reducing wall-clock time for the tool-execution phase from approximately 8 seconds to under 100 milliseconds on cache hits. Estimated per-analysis cost using `gemini-2.5-flash-lite` was under $0.01 per site.

Limitations and Future Directions

The system has meaningful limitations that bound the scope of its claims. Its heuristic assessment is grounded in static content analysis and cannot observe dynamic interface states, animated transitions, or error recovery flows. The in-memory session store and single-process MCP connection prevent horizontal scaling without architectural changes. The absence of streaming means that analyses of large sites risk HTTP timeout under default proxy configurations.

These limitations are not dead ends. The architecture was designed with extensibility in mind: the `CacheService` interface supports a drop-in Redis replacement, the `EvaluationAgent` factory supports new LLM providers with a single additional branch, and LangChain's `astream_events` API provides a clear path to streaming. The most impactful next steps—streaming responses, distributed state management, scheduled analyses with change detection, and an authentication layer—are documented in Chapter 6 as a concrete roadmap.

Broader Implications

Beyond the specific system built here, this thesis demonstrates a broader principle: the ReAct paradigm, combined with purpose-built tool integrations via open protocols like MCP, is capable of automating analytical workflows that have traditionally required sustained human expertise. Website analysis is one instance of a much larger class of tasks—technical auditing, accessibility review, competitive benchmarking, documentation generation—that share the same structure: explore an environment, gather evidence, apply a structured evaluation frame-

work, and produce a report. The Agentic Crawler shows that this structure is amenable to automation at low cost and with useful accuracy, and that the primary remaining barrier is not capability but engineering: streaming, scalability, and integration into existing workflows.

As LLMs continue to improve in context length, reasoning quality, and tool-use reliability, the approximation gap between automated and expert-led evaluation will continue to narrow. The work presented in this thesis is a step towards a future in which systematic usability evaluation is not a scarce, expensive resource but a routine, automated part of every web development workflow.

Bibliography

[1] Anthropic. *Model Context Protocol (MCP): An Open Standard for Connecting AI Assistants to the Systems Where Data Lives*. Technical announcement. 2024. URL: https://www.anthropic.com/news/model-context-protocol.

[2] Agnieszka Bojko. *Eye Tracking the User Experience: A Practical Guide to Research*. Rosenfeld Media, 2013.

[3] Junghoo Cho, Hector Garcia-Molina, and Lawrence Page. "Efficient Crawling Through URL Ordering". In: *Proceedings of the 7th International World Wide Web Conference*. Elsevier, 1998, pp. 161–172.

[4] Emilio Ferrara et al. "Web Data Extraction, Applications and Techniques: A Survey". In: *Knowledge-Based Systems* 70 (2014), pp. 301–323. DOI: 10.1016/j.knosys.2014.07.007.

[5] Erich Gamma et al. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.

[6] Jesse James Garrett. *The Elements of User Experience: User-Centered Design for the Web and Beyond*. 2nd. New Riders, 2011.

[7] Gemini Team and Google. "Gemini: A Family of Highly Capable Multimodal Models". In: *arXiv preprint arXiv:2312.11805* (2023). URL: https://arxiv.org/abs/2312.11805.

[8] Hongliang He et al. "WebAgent: A Long-Context Language Model-Based Agent for Web Automation Tasks". In: *Proceedings of the 2nd Workshop on Agent Learning in Open-Endedness (ALOE) at NeurIPS*. 2023. URL: https://arxiv.org/abs/2307.12856.

[9] LangChain AI. *LangChain: Building Applications with Large Language Models*. Software framework. 2023. URL: https://www.langchain.com.

[10] Mendable AI. *Firecrawl: Crawl and Convert Any Website into LLM-Ready Data*. Web service. 2024. URL: https://www.firecrawl.dev.

[11] Mermaid contributors. *Mermaid: Diagramming and Charting Tool*. Open-source library. 2023. URL: https://mermaid.js.org.

[12] Meta Open Source. *React: The Library for Web and Native User Interfaces*. Open-source library. 2023. URL: https://react.dev.

[13] Jakob Nielsen. "Enhancing the Explanatory Power of Usability Heuristics". In: *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems* (1994), pp. 152–158. DOI: 10.1145/191666.191729.

[14] Jakob Nielsen and Rolf Molich. "Heuristic Evaluation of User Interfaces". In: *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems* (1990), pp. 249–256. DOI: 10.1145/97243.97281.

[15] Christopher Olston and Marc Najork. "Web Crawling". In: *Foundations and Trends in Information Retrieval*. Vol. 4. 3. Now Publishers, 2010, pp. 175–246. DOI: 10.1561/1500000017.

[16] OpenAI. *GPT-4 Technical Report*. Tech. rep. OpenAI, 2023. URL: https://arxiv.org/abs/2303.08774.

[17] Jeff Patton. *User Story Mapping: Discover the Whole Story, Build the Right Product*. O'Reilly Media, 2014.

[18] Pydantic contributors. *Pydantic: Data Validation Using Python Type Hints*. Open-source library. 2023. URL: https://docs.pydantic.dev.

[19] Sebastian Ramirez. *FastAPI: Modern, Fast Web Framework for Building APIs with Python*. Open-source framework. 2023. URL: https://fastapi.tiangolo.com.

[20] react-markdown contributors. *react-markdown: Markdown Component for React*. Open-source library. 2023. URL: https://github.com/remarkjs/react-markdown.

[21] Pranab Sahoo et al. "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications". In: *arXiv preprint arXiv:2402.07927* (2024). URL: https://arxiv.org/abs/2402.07927.

[22] Sander Schulhoff et al. "The Prompt Report: A Systematic Survey of Prompting Techniques". In: *arXiv preprint arXiv:2406.06608* (2024). URL: https://arxiv.org/abs/2406.06608.

[23] Yuxiao Tang, Q. Vera Liao, and Negar Salehi. "Leveraging Large Language Models for Automated Usability Evaluation". In: *Extended Abstracts of the ACM CHI Conference on Human Factors in Computing Systems*. 2023. URL: https://arxiv.org/abs/2307.10455.

[24] Hugo Touvron et al. "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971* (2023). URL: https://arxiv.org/abs/2302.13971.

[25] Russ Unger and Carolyn Chandler. *A Project Guide to UX Design: For User Experience Designers in the Field or in the Making*. 2nd. New Riders, 2012.

[26] Robert A. Virzi. "Refining the Test Phase of Usability Evaluation: How Many Subjects Is Enough?" In: *Human Factors* 34.4 (1992), pp. 457–468. DOI: 10.1177/001872089203400407.

[27] Vite contributors. *Vite: Next Generation Frontend Tooling*. Open-source build tool. 2023. URL: https://vitejs.dev.

[28] Jason Wei et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 24824–24837. URL: https://arxiv.org/abs/2201.11903.

[29] Lilian Weng. *LLM-powered Autonomous Agents*. Blog post, Lil'Log. 2023. URL: https://lilianweng.github.io/posts/2023-06-23-agent/.

[30] Will Willmott. *Rich: A Python Library for Rich Text and Beautiful Formatting in the Terminal*. Open-source library. 2023. URL: https://github.com/Textualize/rich.

[31] Shunyu Yao et al. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations (ICLR)*. 2023. URL: https://arxiv.org/abs/2210.03629.

Appendix A

API Endpoint Reference

Table A.1 provides a complete reference for all REST API endpoints exposed by the Agentic Crawler's FastAPI server. All routes are prefixed with `/api`. The server returns HTTP 503 on any endpoint if the MCP connection has not yet completed initialisation during startup.

Method	Path	Request body	Response fields	Status
`POST`	`/api/chat`	`message` (string, required); `session_id` (string, optional)	`session_id`, `assistant_text`, `tool_events`, `error`, `report_saved_path`	200, 503
`POST`	`/api/session/clear`	`session_id` (string, required)	`ok` (boolean)	200, 503
`GET`	`/api/reports`	—	`reports` (list of strings)	200
`GET`	`/api/outputs`	—	`outputs` (list of strings)	200
`GET`	`/api/stats`	—	`execution` (`ExecutionStats`), `cache` (`CacheStats`)	200, 503
`GET`	`/api/health`	—	`status` (string), `ready` (boolean)	200

Table A.1: Complete REST API endpoint reference.

Request and Response Schemas

`POST /api/chat`

Request body (JSON):

```
{
  "message": "Analyze https://example.com",
  "session_id": "550e8400-e29b-41d4-a716-446655440000"
```

```
}
```

Response body (JSON):

```
{
  "session_id": "550e8400-e29b-41d4-a716-446655440000",
  "assistant_text": "# Website Analysis Report: ...",
  "tool_events": [
    {
      "name": "firecrawl_map",
      "args": {"url": "https://example.com"},
      "cached": false,
      "duration_seconds": 2.41,
      "output": "{\"links\": [...]}",
      "saved_path":
       ↪  "tool_outputs/20260329_firecrawl_map_example.com.json"
    }
  ],
  "error": null,
  "report_saved_path": "reports/Website_Analysis_20260329_example.com.md"
}
```

On the first request, omit `session_id`; the server will generate one and return it in the response. Pass the same value on all subsequent turns to maintain conversation continuity.

GET /api/stats

Returns two nested objects:

```
{
  "execution": {
    "total_calls": 6,
    "input_tokens": 24000,
    "output_tokens": 3200,
    "total_tokens": 27200,
    "estimated_cost": 0.0092,
    "actions_used": 0,
    "outputs_saved": 6
  },
  "cache": {
```

```
    "hits": 0,
    "misses": 6,
    "hit_rate": "0.0%",
    "size_mb": "0.12",
    "entries": 6
  }
}
```

Appendix B

Sample Generated Website Analysis Report

This appendix reproduces the website analysis report generated by the Agentic Crawler for `web-scraping.dev/products`, a mock e-commerce catalogue designed for web scraping practice. The report is presented as produced by the system, with Mermaid diagram source blocks shown as plain text (the rendered diagrams are visible in the browser frontend). Minor formatting adjustments have been made for print layout; emoji section markers from the original output have been omitted.

Website Analysis Report: web-scraping.dev

Executive Summary

- **Website URL:** `https://web-scraping.dev/products`
- **Analysis Date:** 2026-04-13 17:36:43 UTC
- **Languages Detected:** English
- **Total Pages Analysed:** 1
- **Main Sections:** 1
- **Key User Journeys Identified:** 1

Website Summary

The website **web-scraping.dev** serves as a mock e-commerce platform specifically designed for testing web scraping techniques. It provides a variety of mock products that users can scrape, making it an ideal resource for developers and data scientists looking to practice or demonstrate web scraping skills. The target audience includes developers, data analysts, and students interested in learning about web scraping and testing their skills in a controlled environment. The business model revolves around offering a free-to-use platform for educational purposes.

Content Overview

The main content on the products page includes:

- **Product Categories:** Apparel, Consumables, Household.
- **Product Listings:** Each product includes a name, description, price, and thumbnail image.
- **Pagination:** 28 results across 6 pages.

Key content themes include mock products for testing scraping techniques and gaming-related consumables (energy potions). Products are organised into categories, each with a dedicated section including images, descriptions, and pricing.

Sitemap Diagram

```
graph TD
    A["Homepage"] --> B["Products"]
    B --> C["Product 1: Box of Chocolate Candy"]
    B --> D["Product 2: Dark Red Energy Potion"]
    B --> E["Product 3: Teal Energy Potion"]
    B --> F["Product 4: Red Energy Potion"]
    B --> G["Product 5: Blue Energy Potion"]
```

User Flow Diagram: User Browsing Products

```
flowchart TD
    Start(["User Lands on Products Page"]) --> ViewProducts["Views
     ↪  Product Listings"]
    ViewProducts --> ClickProduct1["Clicks on 'Box of Chocolate Candy'"]
    ClickProduct1 --> ViewDetails1["Views Product Details"]
```

```
ViewDetails1 --> AddToCart1["Adds to Cart"]
AddToCart1 --> ContinueShopping["Continues Browsing"]
ContinueShopping --> ViewProducts
```

Site Structure Details

- **Homepage** (`/`): Main entry point for the website.
- **Products** (`/products`): Displays a list of mock products for scraping.
  - Product 1 (`/product/1`): Box of Chocolate Candy
  - Product 2 (`/product/2`): Dark Red Energy Potion
  - Product 3 (`/product/3`): Teal Energy Potion
  - Product 4 (`/product/4`): Red Energy Potion
  - Product 5 (`/product/5`): Blue Energy Potion

Key User Journeys

Browsing Products. Users land on the products page, view various products, click on individual items to see details, and can add them to their cart. Steps: landing on products page → viewing listings → clicking a product → viewing details → adding to cart.

Navigation Patterns

Primary navigation is via direct access to the products page from the homepage. Pagination controls allow browsing through multiple pages of products. No search functionality is present on the analysed page.

Content Types & Features

Five mock products are displayed with images and descriptions. Interactive elements include links to product details and pagination for browsing through multiple pages.

Design & UX Observations

The design is simple and functional, focusing on product display. The colour scheme is predominantly blue and white with product images providing contrast. Typography is clear and legible. Products are arranged in a grid-like layout that appears adaptable for mobile devices.

Heuristic Evaluation

Heuristic	Rating	Evidence	Impact	Recommendation
Visibility of system status	Pass	Page loads quickly with clear product listings.	Users can easily see available products.	N/A
Match between system and the real world	Pass	Product names and descriptions are relatable and clear.	Users can easily understand the offerings.	N/A
User control and freedom	Partial	Users can navigate back but lack a clear homepage link.	Users may feel slightly lost.	Add a clear link to the homepage.
Consistency and standards	Pass	Consistent layout and design across product listings.	Users can predict how to interact.	N/A
Error prevention	Fail	No confirmation when adding items to the cart.	Users may accidentally add multiple items.	Implement a confirmation message.
Recognition rather than recall	Pass	Product images and descriptions aid recognition.	Users can easily identify products.	N/A
Flexibility and efficiency of use	Partial	Pagination available but no category filters.	Tedious to browse many pages.	Introduce filtering options.
Aesthetic and minimalist design	Pass	Clean design focused on products.	Users can focus without distractions.	N/A
Help users recognise errors	Fail	No error messages for actions like adding to cart.	Users may be confused if an action fails.	Implement error messages.
Help and documentation	Fail	No help or documentation available.	Users may struggle with site usage.	Provide a help section or FAQ.

Closing Summary. The website performs well in terms of usability but has areas for improvement in user control and error prevention. Top strengths: clear product listings, consistent design, effective recognition aids. Top issues: lack of homepage navigation, absence of confirmation messages, no filtering options. Most critical priorities: add a homepage link, implement confirmation messages for cart actions, and introduce category filtering.

External Integrations

No payment processors, analytics tools, third-party widgets, social media integrations, or API integrations were detected. This is expected given the site's nature as a mock platform.

Technical Observations

The website appears to be a simple HTML/CSS site with no complex frameworks detected. The page loads quickly. Proper meta tags and structured data are present. Basic accessibility features exist but could be improved. The site uses HTTPS.

Additional Notes

Product descriptions are engaging and informative. The overall user experience is straightforward, though navigation improvements are needed. As a mock site, it serves a niche market for web scraping practice. Recommendations include enhancing navigation options, adding confirmation messages, and implementing a help section.

Appendix C

Environment Variables and Deployment

Backend Environment Variables

All backend configuration is supplied via a `.env` file placed in the `backend/` directory. Copy `.env.example` to `.env` and fill in the required values before running the application. Table C.1 lists all supported variables.

Variable	Purpose	Required	Default
`FIRECRAWL_API_KEY`	Firecrawl web-crawling service key	Yes	—
`GOOGLE_API_KEY`	Google Gemini LLM API key	One of these	—
`GROQ_API_KEY`	Groq LLaMA inference API key	One of these	—
`DEBUG_MODE`	Enable verbose logging	No	`false`
`ENABLE_CACHING`	Enable LRU tool-call cache	No	`true`
`MODEL_TEMPERATURE`	LLM sampling temperature	No	`0.3`
`CACHE_TTL_HOURS`	Cache entry time-to-live	No	`24`
`MAX_CACHE_SIZE`	Maximum cache size (MB)	No	`100`
`CORS_ORIGINS`	Allowed CORS origins	No	`*`
`AGENTIC_CRAWLER_HOST`	API server bind address	No	`127.0.0.1`
`AGENTIC_CRAWLER_PORT`	API server port	No	`8000`
`AGENTIC_CRAWLER_RELOAD`	Enable uvicorn auto-reload	No	`false`

Table C.1: Backend environment variable reference.

Frontend Environment Variables

The frontend reads a single variable from a `.env` file in the `frontend/` directory:

```
VITE_API_BASE_URL=http://127.0.0.1:8000
```

This value is injected at build time via `import.meta.env.VITE_API_BASE_URL` and must match the host and port on which the backend server is running. No trailing slash should be included.

Deployment Instructions

Prerequisites

- Python 3.11 or later
- Node.js 18 or later (required for the Firecrawl MCP server, launched automatically via `npx firecrawl-mcp`)
- A Firecrawl API key and at least one of a Google Gemini or Groq API key

Backend: CLI

```
# 1. Navigate to the backend directory
cd backend

# 2. Install Python dependencies
pip install -e .
# or, using uv:
uv pip install -e .

# 3. Create and populate the .env file
cp .env.example .env
# Edit .env and add your API keys

# 4. Run the CLI
python main.py
```

Type a URL to analyse, e.g. `Analyze https://example.com`. Use `/help` to list available slash commands.

Backend: HTTP API Server

```
# After completing steps 1-3 above:

# 5. Start the FastAPI server
```

```
agentic-crawler-web
# or equivalently:
uvicorn agentic_crawler.web.app:app --host 127.0.0.1 --port 8000
```

The server will be available at `http://127.0.0.1:8000`. Interactive OpenAPI documentation is served at `http://127.0.0.1:8000/docs`.

Frontend

```
# 1. Navigate to the frontend directory
cd frontend

# 2. Install Node.js dependencies
npm install

# 3. Create and populate the .env file
cp .env.example .env
# Set VITE_API_BASE_URL to match the backend server address

# 4. Start the development server
npm run dev

# Or build a production bundle:
npm run build
# Then serve the dist/ directory with any static file server.
```

Appendix D

Project File Structure

The repository at `https://github.com/SOURON/thesis-work` is organised into two top-level directories: `backend/` containing the Python application, and `frontend/` containing the React SPA. The annotated tree below lists every file and directory of significance.

```
thesis-work/
+-- backend/
|   +-- src/
|   |   +-- agentic_crawler/
|   |       +-- agents/
|   |       |   +-- evaluation_agent.py  # EvaluationAgent: LLM + MCP
↪  tools
|   |       |   +-- prompts.py           # Six-phase system prompt
|   |       +-- config/
|   |       |   +-- settings.py          # Config singleton, env loading
|   |       +-- models/
|   |       |   +-- execution.py         # ToolCall, ExecutionStats,
↪  CacheStats
|   |       +-- services/
|   |       |   +-- cache_service.py     # LRU + TTL CacheService
|   |       |   +-- chat_service.py      # ChatSessionService,
↪  SessionStore,
|   |       |   |                        #   TurnResult, ToolEvent
|   |       |   +-- content_normalize.py # Multimodal content
↪  normalisation
|   |       |   +-- tracker_service.py   # ExecutionTracker
|   |       +-- utils/
|   |       |   +-- display_utils.py     # Rich terminal formatting
```

```
|   |       |   +-- file_utils.py        # Report and output file
↪  persistence
|   |       |   +-- mcp_utils.py         # MCP tool loading helpers
|   |       +-- web/
|   |       |   +-- app.py               # FastAPI app, lifespan, CORS
|   |       |   +-- routes.py            # REST endpoints (/api/*)
|   |       |   +-- main.py              # uvicorn entry point
|   |       +-- cli.py                   # AgenticCrawlerCLI, slash
↪  commands
|   +-- tests/                           # Unit test suite
|   +-- reports/                         # Generated analysis reports
↪  (runtime)
|   +-- tool_outputs/                    # Raw Firecrawl outputs
↪  (runtime)
|   +-- cache/                           # LRU cache storage (runtime)
|   +-- sessions/                        # Session data (runtime)
|   +-- main.py                          # CLI entry point
|   +-- pyproject.toml                   # Package metadata and
↪  dependencies
|   +-- requirements.txt                 # Pinned dependency list
|   +-- .env.example                     # Environment variable template
|   +-- README.md                        # Backend usage documentation
|
+-- frontend/
    +-- src/
    |   +-- api/
    |   |   +-- client.ts                # Typed fetch wrappers for all
     ↪  endpoints
    |   |   +-- types.ts                 # ChatRequest, ChatResponse,
     ↪  ToolEvent
    |   +-- components/
    |   |   +-- MarkdownMessage.tsx      # react-markdown renderer +
     ↪  Mermaid router
    |   |   +-- MermaidBlock.tsx         # Async Mermaid SVG renderer
    |   |   +-- ToolEventsPanel.tsx      # Collapsible tool event display
    |   +-- App.tsx                      # Root component, all global
     ↪  state
    |   +-- App.css                      # Layout, theming, dark mode
```

```
|   +-- config.ts                    # VITE_API_BASE_URL,
 ↪  SESSION_STORAGE_KEY
|   +-- main.tsx                     # React DOM entry point
+-- public/                          # Static assets
+-- dist/                            # Production build output
 ↪  (generated)
+-- package.json                     # Node.js dependencies
+-- vite.config.ts                   # Vite build configuration
+-- tsconfig.json                    # TypeScript configuration
+-- .env.example                     # VITE_API_BASE_URL template
+-- README.md                        # Frontend usage documentation
```

Runtime Directories

The `backend/` runtime directories are created automatically on first run if they do not exist:

- `reports/` — Markdown website analysis reports, named `Website_Analysis_{timestamp}_{url}.md`.
- `tool_outputs/` — JSON records of individual Firecrawl tool calls, named `{timestamp}_{tool}_{url}.json`.
- `cache/` — Reserved for future persistent cache storage; the current LRU cache is held entirely in memory.
- `sessions/` — Reserved for future persistent session storage; the current `SessionStore` is held entirely in memory.

Appendix E

System Prompt

This appendix reproduces the complete system prompt passed to the `EvaluationAgent` at construction time, as defined in `backend/src/agentic_crawler/agents/prompts.py`. The prompt engineering decisions that shaped this prompt are discussed in Section 2.4.

Note that the `[date]` placeholder in the Executive Summary template is replaced at runtime with the current UTC timestamp by the `get_system_prompt()` function. The binding rule that enforces verbatim reproduction of this date is injected programmatically and is not shown below.

```
You are an expert Website Analyst with deep knowledge of web
architecture, user experience, and information architecture. Your job
is to ANALYZE websites thoroughly and provide comprehensive website
summaries, content overviews, user flow diagrams, and sitemap diagrams.

**Your Analysis Approach (Step-by-Step):**

STEP 1: DISCOVER THE WEBSITE
- Use firecrawl_map to discover all pages and site structure
- Scrape the homepage first to understand the business
- Identify key pages (contact, features, pricing, about, etc.)
- Map out all navigation paths and page relationships

STEP 2: ANALYZE STRUCTURE & CONTENT
- What does this website DO? (Purpose, target users, business model)
- What are the main features/functionalities?
- What content types are present? (blog, products, services,
  documentation)
- What are the key user journeys?
- How is the site organized? (hierarchical structure, categories,
```

sections)

STEP 3: IDENTIFY USER FLOWS
- What are the primary user goals?
- What paths do users take to achieve these goals?
- What are the entry points? (homepage, landing pages, search)
- What are the conversion points? (signup, purchase, contact, download)
- What decision points exist? (product selection, plan selection, etc.)

STEP 4: CREATE DIAGRAMS
- Generate a sitemap diagram showing all pages and their relationships
- Generate user flow diagrams for key user journeys
- Use Mermaid syntax for all diagrams
- In every diagram, wrap human-readable label text in double quotes inside node shapes and on edges (see Mermaid Diagram Guidelines)

Output Format for Website Analysis Reports:
When asked to analyze/report on a website, respond with raw Markdown (headings, lists, tables, bold, etc.) as the message body itself.

Do not fence the whole report: Never wrap the entire answer in a ```markdown code block. Begin directly with the first heading. Use fenced blocks only for Mermaid diagrams, not for the full document.

Sections -- no extras: Use only the sections defined below (the ## and ### headings shown in this template -- same titles and order). Do not add other top-level ## sections. Do not add parallel summaries outside this outline. If something does not fit, place it inside the closest existing section or subsection.

Website Analysis Report: [Website Name]

Executive Summary
- **Website URL**: [url]
- **Analysis Date**: [date]
- **Languages Detected**: [list]
- **Total Pages Analyzed**: [number]
- **Main Sections**: [number]
- **Key User Journeys Identified**: [number]

```
## Website Summary
[Comprehensive description of what the website does, target audience,
main purpose, business model, and value proposition.]

## Content Overview
[Detailed overview of all content types and sections found.]

## Sitemap Diagram
[Mermaid graph TD showing the complete site structure.]

## User Flow Diagrams
[Mermaid flowchart TD diagrams for each key user journey.
Include decision points, actions, and outcomes.]

## Site Structure Details
[Page-by-page URL breakdown with purposes.]

## Key User Journeys
[List and describe the main user journeys identified.]

## Navigation Patterns
[Primary, secondary, footer, breadcrumbs, search.]

## Content Types & Features
[All content types and interactive features found.]

## Design & UX Observations
[Design patterns, UI components, and UX observations.]

## Heuristic Evaluation
[Evaluate the website against Nielsen's ten usability heuristics.]

Evaluation Rules:
- Assess each heuristic using Pass / Partial / Fail.
- You must evaluate all ten heuristics -- the table must have
  exactly ten data rows (one per heuristic), in the same order
  as the numbered list.
- Keep every judgment grounded in concrete evidence.
- Reference specific pages, UI elements, or user flows.
```

```
- Distinguish between minor friction and major usability breakdowns.

Required Output Matrix:
| Heuristic name | Rating | Evidence | Impact | Recommendation |

Nielsen's Ten Usability Heuristics:
1.  Visibility of system status.
2.  Match between system and the real world.
3.  User control and freedom.
4.  Consistency and standards.
5.  Error prevention.
6.  Recognition rather than recall.
7.  Flexibility and efficiency of use.
8.  Aesthetic and minimalist design.
9.  Help users recognize, diagnose, and recover from errors.
10. Help and documentation.

Closing Summary: Overall evaluation summary, top 3 strengths,
top 3 issues, most critical improvement priorities.

## External Integrations
[External services or integrations found.]

## Technical Observations
[Technology stack, performance, SEO, accessibility, security.]

## Additional Notes
[Any other relevant observations.]

**When to Use Tools:**

firecrawl_map:    ALWAYS use FIRST to discover site structure.
                  url: required.

firecrawl_scrape: Scrape individual important pages.
                  url: required,
                  formats: ["markdown"],
                  onlyMainContent: true

firecrawl_crawl:  Bulk crawl multiple pages.
```

```
                  url: required, limit: 10

**Analysis Strategy:**

1. START:      Use firecrawl_map to get all URLs
2. PRIORITIZE: Identify most important pages
3. SCRAPE:     Scrape each important page individually
4. ANALYZE:    Understand purpose, content, navigation, journeys
5. DIAGRAM:    Create sitemap and user flow diagrams
6. DOCUMENT:   Create comprehensive summary and content overview

**Mermaid Diagram Guidelines:**

Always put visible text inside double quotes within Mermaid syntax.
Use forms like A["Homepage"], B["/products/item-1"],
C("User clicks Buy"), D{"Logged in?"}, and quoted edge labels
such as -->|"Yes"| or -->|"No"|. Quote labels even when they
contain URLs, slashes, parentheses, apostrophes, or multiple words
-- this prevents parse errors and keeps diagrams valid.

**DO:**
  Map the entire site structure first (firecrawl_map)
  Identify all user journeys and goals
  Create valid Mermaid diagrams
  Include actual discovered URLs in Site Structure Details
  Wrap all node and edge label strings in double quotes
  Emit raw Markdown only (no outer ```markdown wrapper)
  Include all 10 rows in the heuristic evaluation table
  Keep only the sections named in this template

**DO NOT:**
  Add sections that are not specified in this template
  Truncate the heuristic table to fewer than 10 data rows
  Wrap the full report in a ```markdown fenced code block
  Just dump raw data
  Skip the discovery phase (map first!)
  Create generic diagrams
  Use invalid Mermaid syntax
  Ignore content details
  Be vague about site structure
```

The prompt above is a faithful reproduction of the production system prompt with minor whitespace adjustments for readability within the page margins. The section-level placeholder descriptions have been abbreviated; the production version includes longer guidance text for each section. The dynamic analysis date binding (replacing `[date]` with the server's UTC timestamp and injecting a verbatim-reproduction rule) is performed programmatically by `get_system_prompt()` and is not shown above. The full source can be inspected at `https://github.com/SOURON/thesis-work/blob/main/backend/src/agentic_crawler/agents/prompts.py`.